

IMS Hands-On Lab

z/OS Connect EE V3, Rest APIs – OpenAPI2 the IMS Service Provider

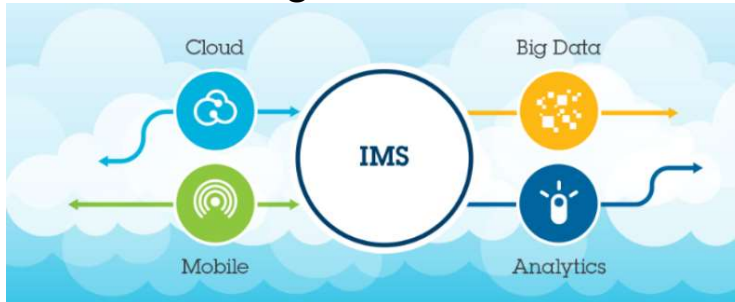
Introduction:

This is an opportunity to get your hands dirty and play with RESTful APIs. Lots of fun and no harm done. This hands-on lab uses the latest z/OS Connect EE V3 to discover, model, enable, and publish services and associated APIs based on trusted IMS transactions. Alternatively, you can choose a lab to see how easily a distributed environment can access IMS mainframe data.

Userid: USER1

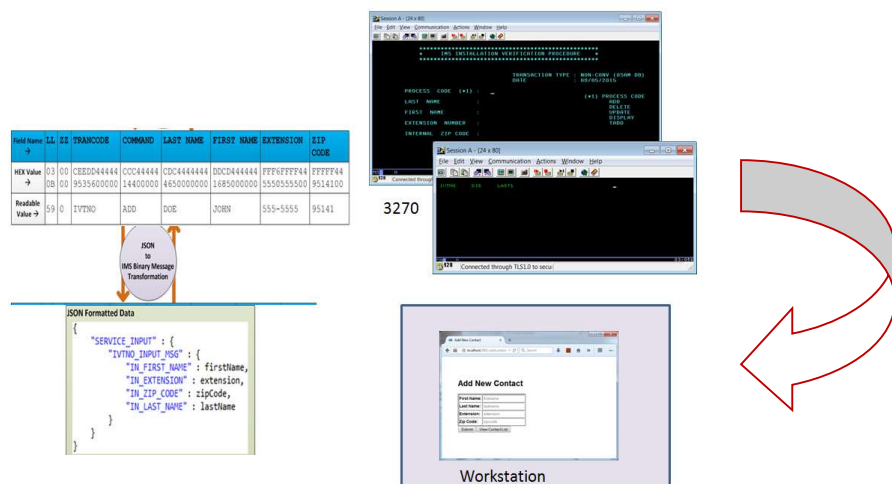
Password: USER1

Part1 - Creating an IMS Transaction Service and API



The goal of this lab is to provide an overview of the steps necessary to:

1. To take **an existing IMS transaction** and **expose the transaction as a set of RESTful services** using z/OS Connect with the IMS Service Provider
2. Create an API from the service.



3. Test the API

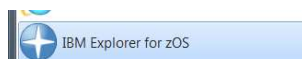
Define, deploy and test RESTful services that drive IMS transactions

1A. Launch z/OS Connect API toolkit

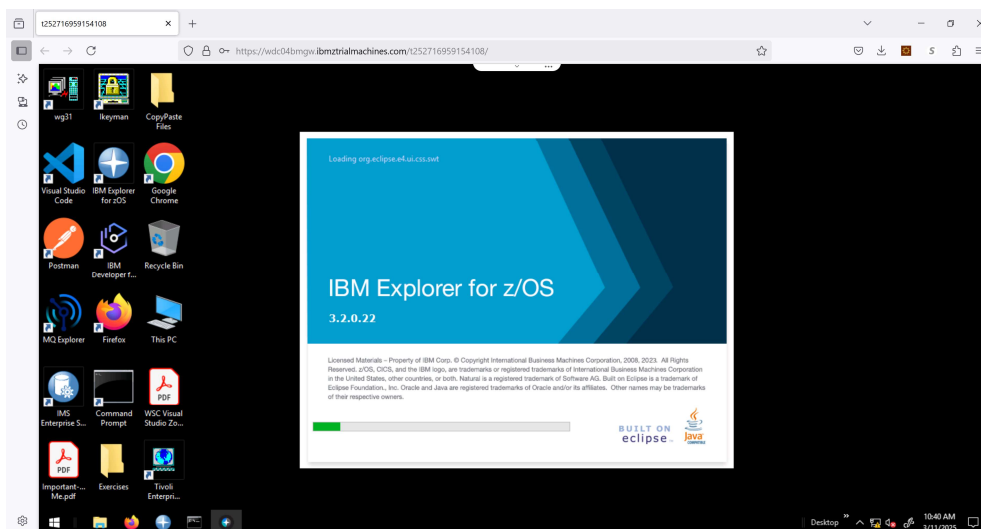
1. Start **z/OS Connect EE API toolkit**
 - a. **Double-click the IBM Explorer for z/OS** blue application icon in the **IMS** and **IMS Tools** folder on the desktop

Or

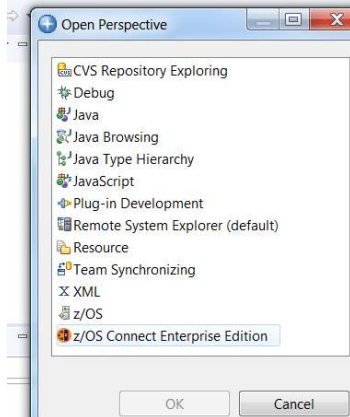
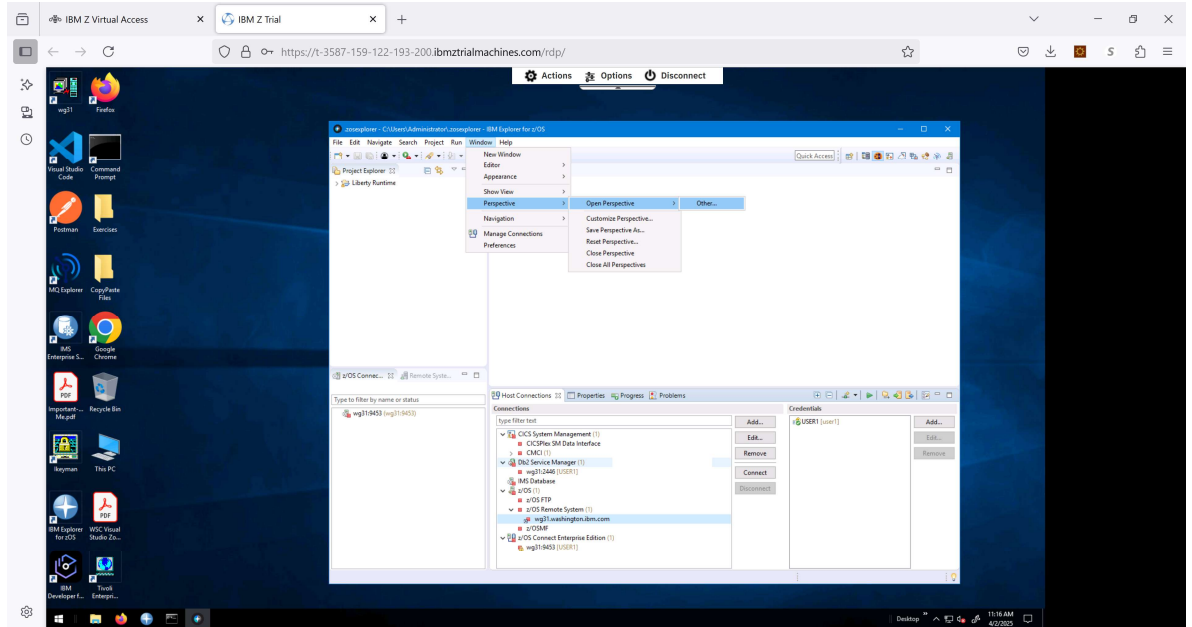
Click on **Start > All Programs > IBM Explorer for z/OS**



or look for this icon on the desktop



2. Open the **z/OS Connect Enterprise Edition** perspective
 - a. From the main menu click **Window > Perspective> Open Perspective > Other**



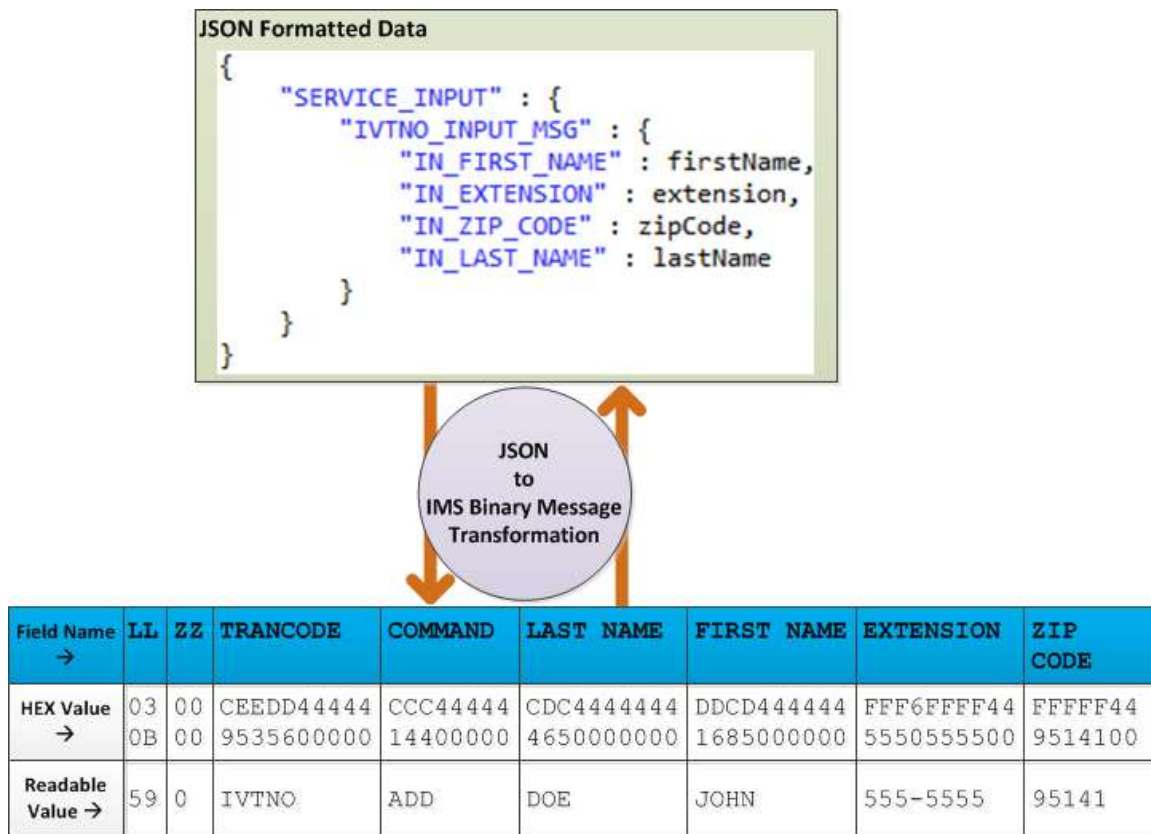
b. ... and then select **z/OS Connect Enterprise Edition**.

A perspective is a pre-defined collection of views that support a specific set of functions for a developer. In this lab, the **z/OS Connect Enterprise Edition** perspective provides a set of views that can be used to develop RESTful services (that invoke IMS transactions) and deploy them to zOS Connect/WebSphere Liberty server.

If you are asked if you want to open up the IMS Explorer perspective, click on No or cancel.

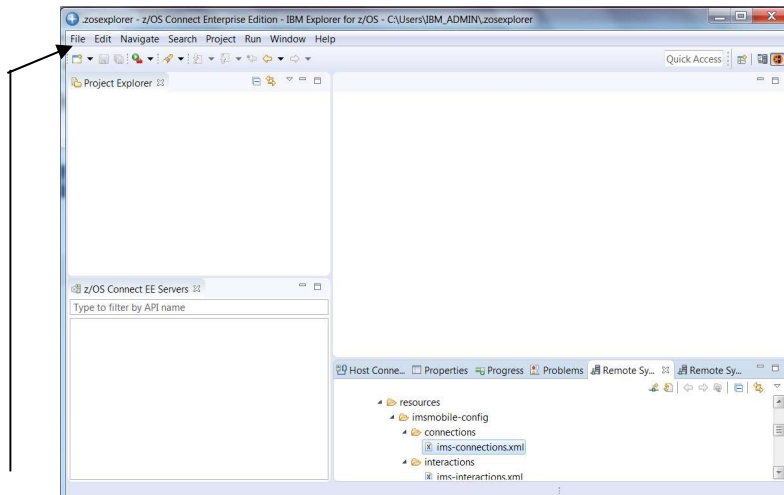
Define the message metadata for the IMS transaction IVTNO

The IMS Service Provider uses message metadata to convert from the RESTful service input, which is in JavaScript Object Notation (JSON) to IMS transaction input, which is an array of structured bytes, defined by the IMS application. Below is a graphic of JSON input and what the transformed IMS data would look like.

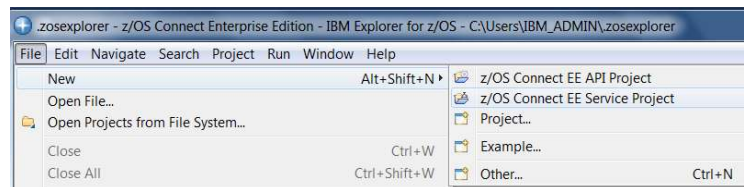


Importing your IMS application's COBOL or PL/I copybook structures for the transaction's input and output messages produces the necessary metadata for the IMS Service Provider to perform this data transformation.

In the following steps, you will import the input and output metadata for IVTNO transaction from a COBOL copybook called DFSIVTNO.cpy. This copybook was previously FTPed down from our application source libraries.

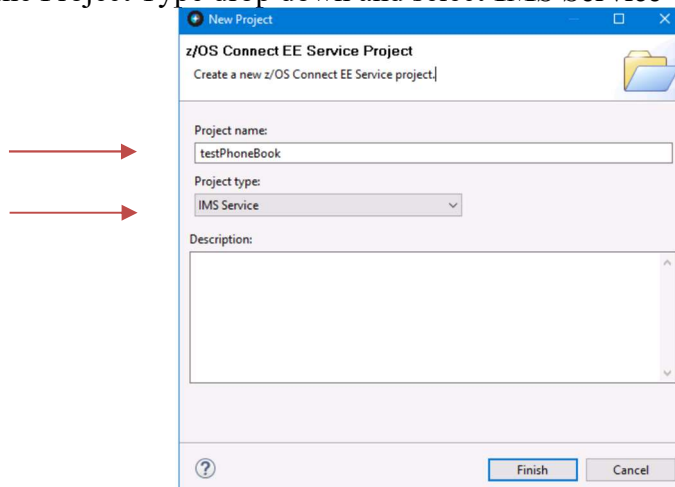


Select **File > New > Project**. The New Project wizard opens select **z/OS Connect EE Service Project**



The z/OS Connect Service Project **New Project** window opens. Provide a project name for example **TestPhoneBook**

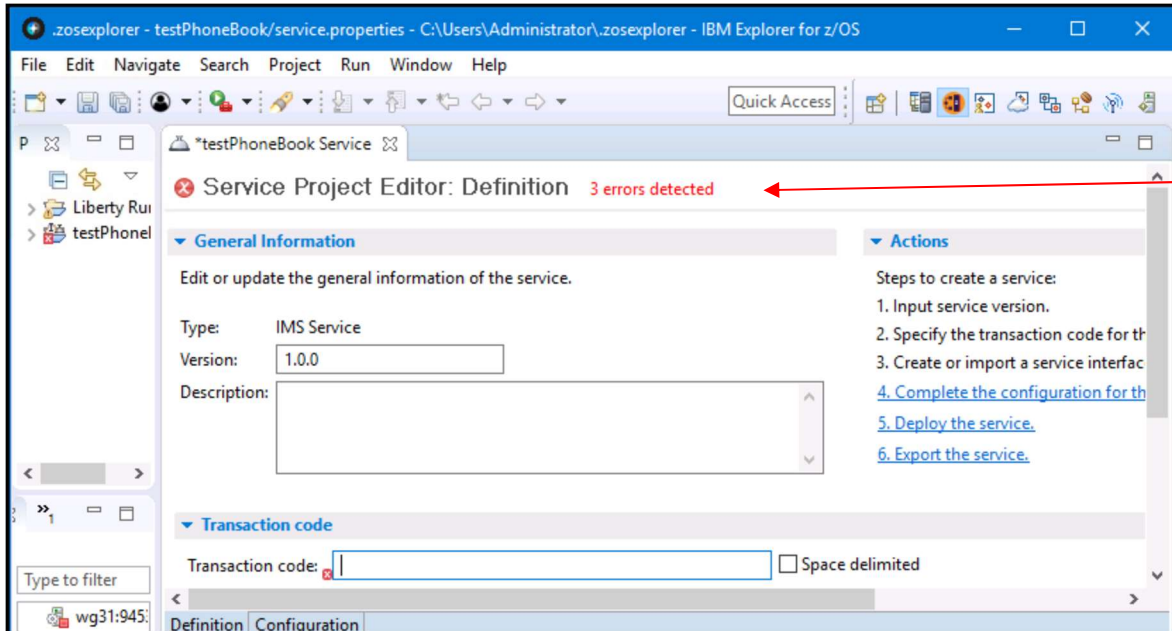
Use the Project Type drop down and select **IMS Service**



Click **Finish**.

You have created an IMS service project in your workspace. You will now define the message metadata and service interfaces for the IMS phonebook application (transaction IVTNO). **Note: The z/OS Connect EE service editor dialog opens, with**

some **errors** initially that indicate some required information is still missing.



You will create one service interface for the request message and another for the response message by importing one or more data structure files (COBOL copybooks or PL/I include).

For reference, the COBOL copybook source contains the following data structures:

```

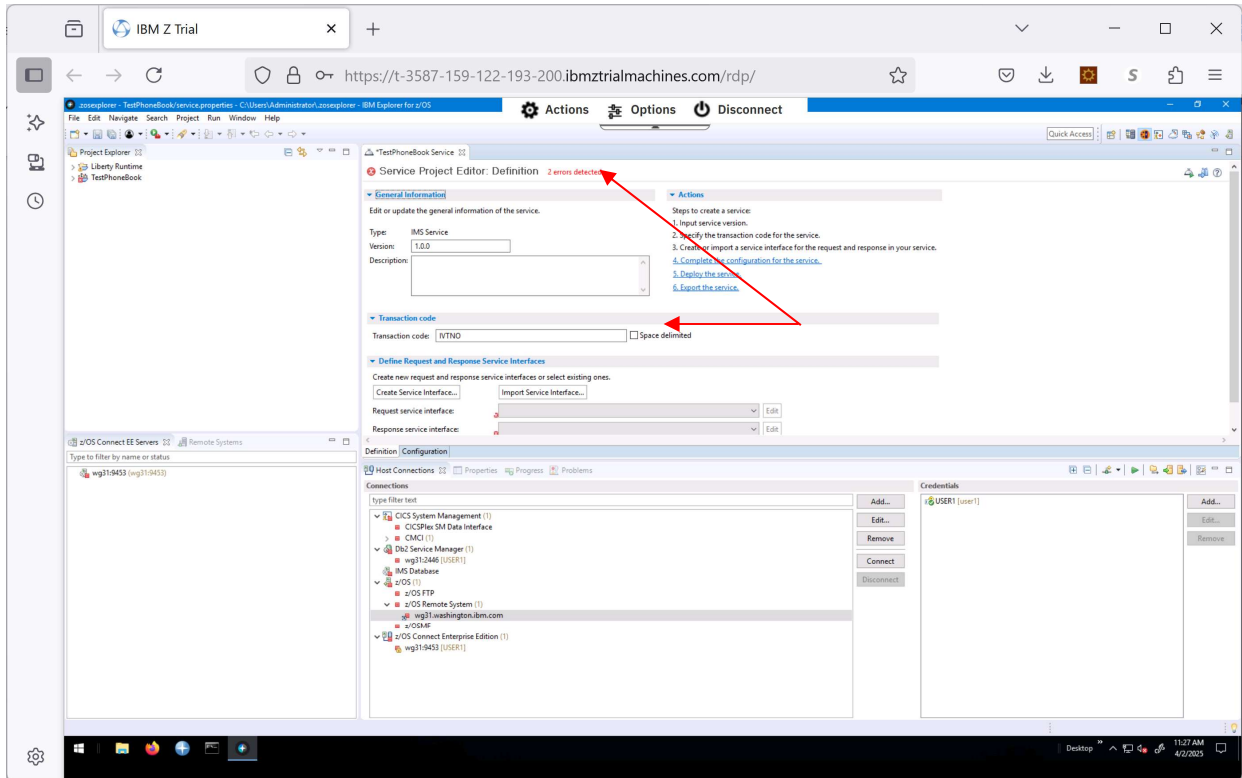
1  * DATA AREA FOR TERMINAL INPUT
2
3  01  IVTNO-INPUT-MSG.
4      02  IN-LL          PICTURE S9(3) COMP.
5      02  IN-ZZ          PICTURE S9(3) COMP.
6      02  IN-TRANCDE     PICTURE X(10) .
7      02  IN-COMMAND     PICTURE X(8) .
8      02  IN-LAST-NAME   PICTURE X(10) .
9      02  IN-FIRST-NAME  PICTURE X(10) .
10     02  IN-EXTENSION    PICTURE X(10) .
11     02  IN-ZIP-CODE     PICTURE X(7) .
12
13  * DATA AREA OUTPUT
14  01  IVTNO-OUTPUT-MSG.
15     02  OUT-LL          PICTURE S9(3) COMP VALUE +95.
16     02  OUT-ZZ          PICTURE S9(3) COMP VALUE +0.
17     02  OUT-MESSAGE     PIC X(40) .
18     02  OUT-COMMAND     PIC X(8) .
19     02  OUT-LAST-NAME   PIC X(10) .
20     02  OUT-FIRST-NAME  PIC X(10) .
21     02  OUT-EXTENSION    PIC X(10) .
22     02  OUT-ZIP-CODE     PIC X(7) .
23     02  OUT-SEGMENT-NO  PICTURE X(4) VALUE '0001'.
24

```

You will use INPUT-MSG and OUTPUT-MSG for the RESTful service that you will create.

In the service project editor, take the following steps:

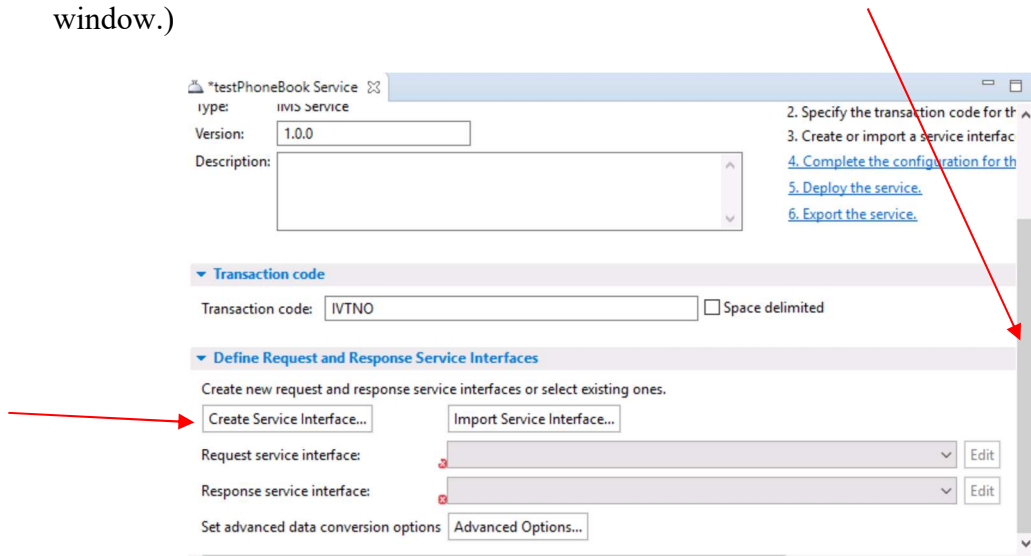
1. For the transaction code to invoke, specify **IVTNO**.



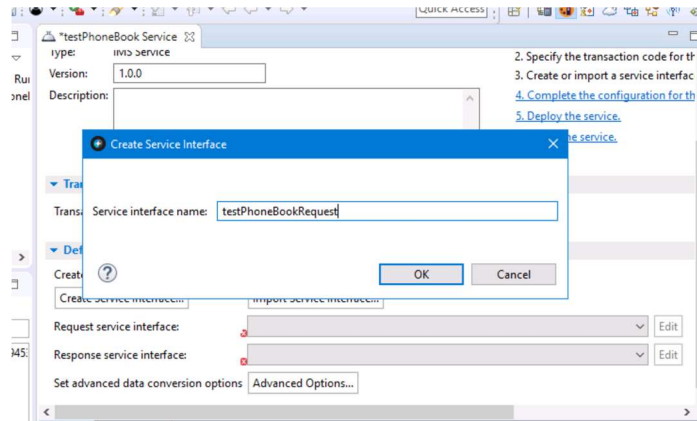
(note that after defining the Transaction code, the error count has gone down to 2)

Now create the service interface for the request and response message. This step involves importing the COBOL or PL/I data structure or a full program into this project and specify, for input and output messages, which fields are to be exposed.

2. In the Define Request and Response Service Interfaces section, click Create Service Interface. (NOTE: you may need to slide the bar down on the right side of the window.)

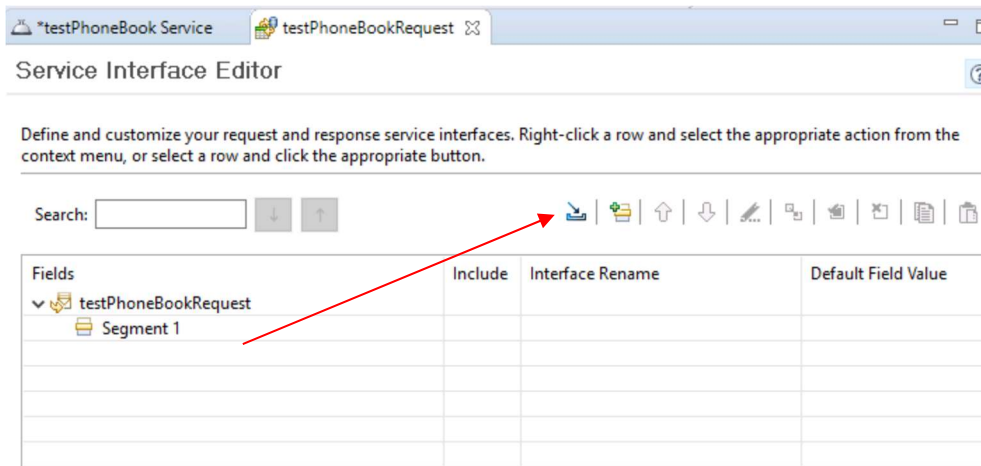


We will first create a service interface for the request messages

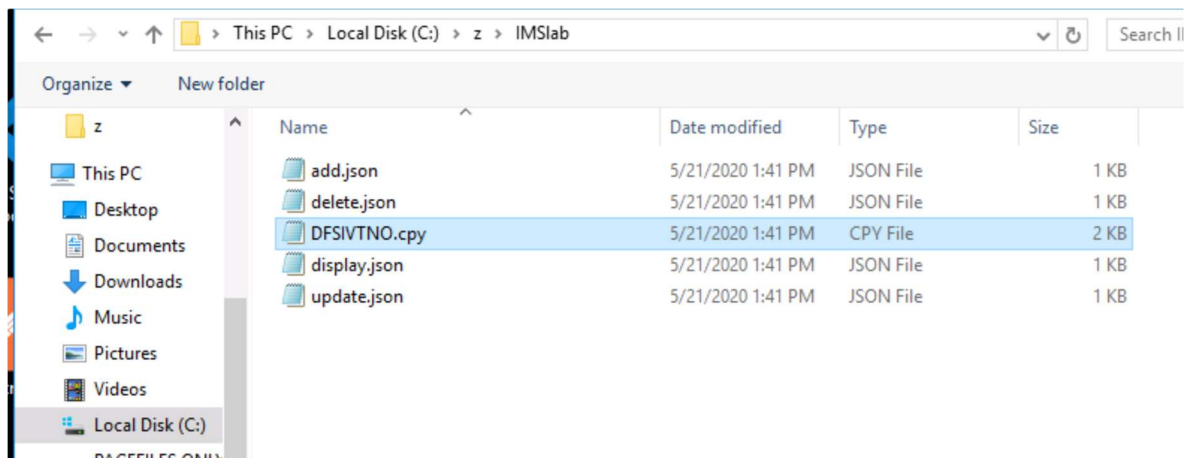


- Specify a Service interface name: such as **testPhoneBookRequest**. This service interface can later be identified as the request service interface for this service project. It can also be reused by other service projects. Click **OK**.

The Service Interface Definition window opens up.



- Click the  button to import the phonebook copybook.
- In the Import wizard, import the input and output message data structures.
 - a. Select where you are importing the phonebook copybook from. For this exercise, the phonebook copybook is on your local system, select Local file system.
 - b. Select the file type. For this tutorial, the default of COBOL data structures only is what we need.
 - c. For the Data structure file field, click Browse.
 - d. In the Select File dialog, navigate to the phonebook copybook file **DFSIVTNO.cpy** on your workstation (**C:\z\IMSlab**) and click Open.



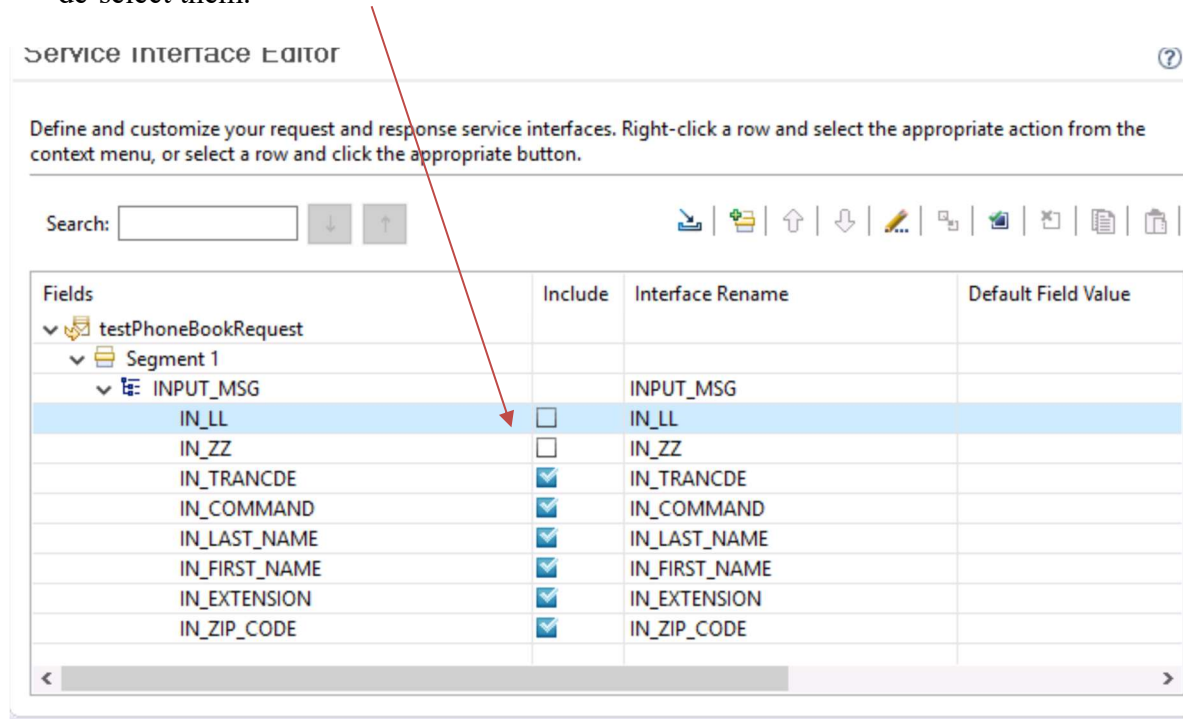
- Specify the data structure for the input messages:

- In the Data structure name list, select INPUT_MSG.
- For the Message name list, `<service_interface_name>` is automatically selected.
- For the Segment name list, ensure that Segment 1 is selected.
- Click Add to Import List to add this message data structure to the Import list.
- Click OK. The service interface editor opens.

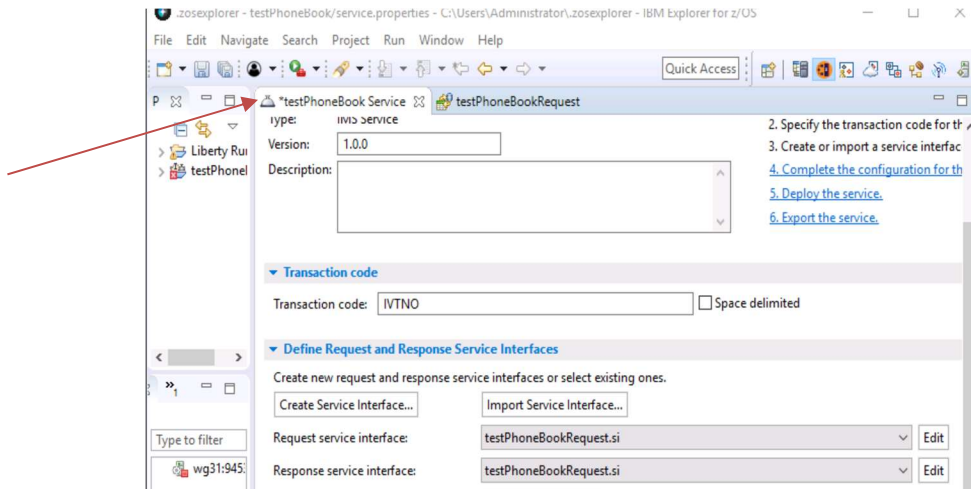
Fields	Include	Interface Rename	Default Field Value
testPhoneBookRequest			
Segment 1			
INPUT_MSG		INPUT_MSG	
IN_LL	☒	IN_LL	
IN_ZZ	☒	IN_ZZ	
IN_TRANCDE	☒	IN_TRANCDE	
IN_COMMAND	☒	IN_COMMAND	
IN_LAST_NAME	☒	IN_LAST_NAME	
IN_FIRST_NAME	☒	IN_FIRST_NAME	
IN_EXTENSION	☒	IN_EXTENSION	
IN_ZIP_CODE	☒	IN_ZIP_CODE	

- Expand Segment 1 and INPUT_MSG to see all fields in the data structure.
- The phonebook service requires most of the fields for input messages, except IN_LL and IN_ZZ. These are IMS internal fields.

- a. Exclude IN_LL and IN_ZZ by clicking the check boxes in the Include column to de-select them.



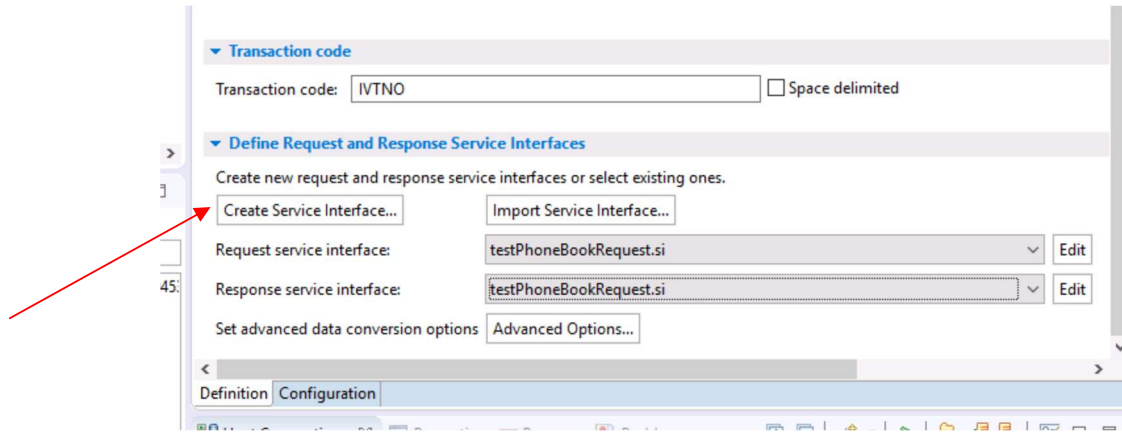
- b. Click **File>Save** to save the **request service** interface.
- c. Click the tab with your service project name to go to the overview page of your service project.



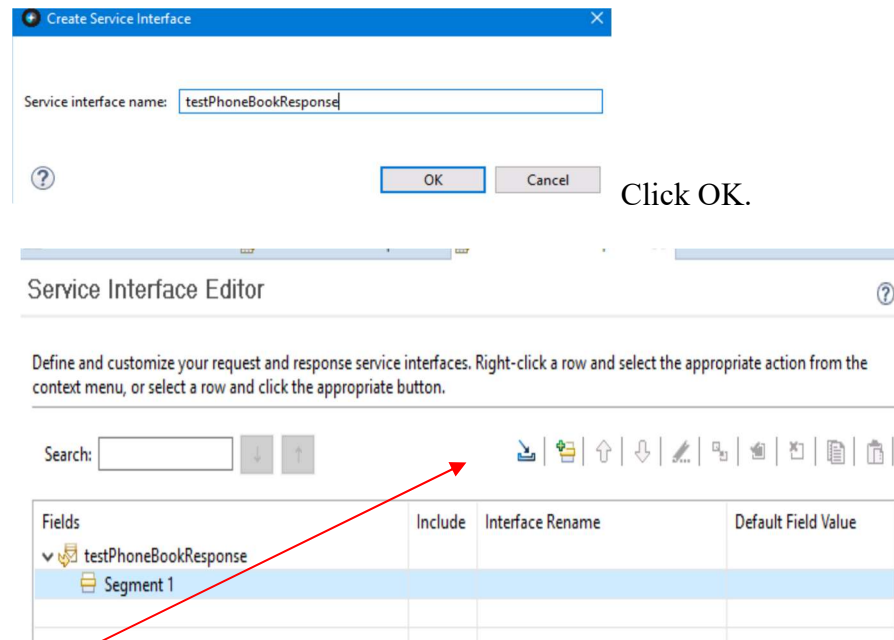
Note that both the Request and Response service interface names are the same at this point.

3. Next define the service interface for the Response message for the phonebook application. (repeat the process you just completed).

In the **Define Request and Response Service Interfaces** section, click **Create Service Interface**.



- Specify a name such as testPhoneBookResponse for the service interface name.



- Click the  button to import the phonebook copybook.

- Select where you are importing the phonebook copybook from. Again, use Local file system: C > z > IMSlab
- Select the file type. For this tutorial, the default of COBOL data structures only is what we need.
- For the Data structure file field, click Browse and select DFSIVTNO
- For the Data structure name, this time select **OUTPUT_AREA**
- Select Add to Import List

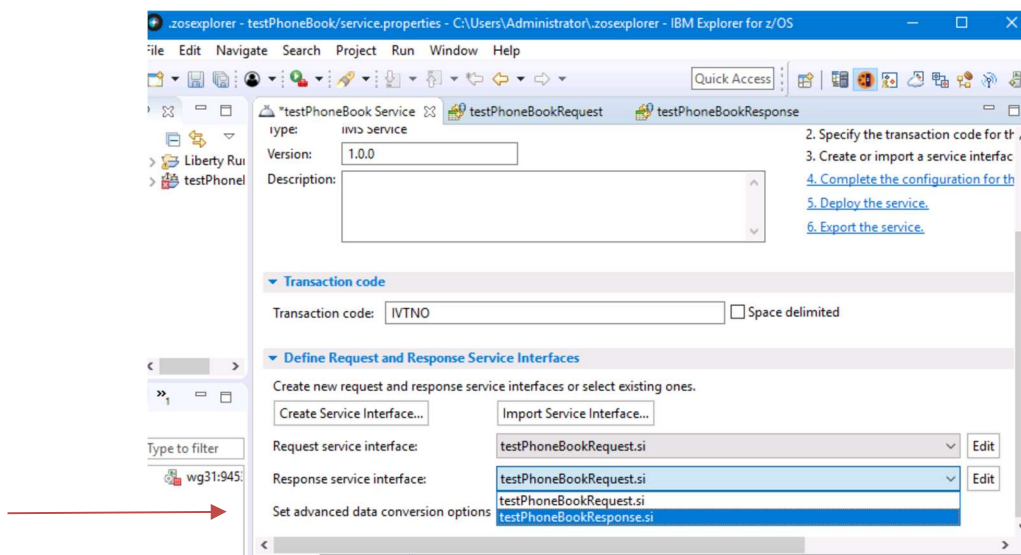
The phonebook service requires most of the fields for output messages, except OUT_LL and OUT_ZZ. These are IMS internal fields.

- Exclude OUT_LL and OUT_ZZ by clicking the check boxes in the Include column to deselect them. Save your changes (select **File > Save**) You have created two service interface files. In the Project Explorer view, two service interface files (.si files) are created in the service-interfaces/ folder in your service project.

The next step is to go back to the service project editor to select this service interface file as the response service interface for this project. Go back to the service project editor by clicking the tab that is named after your service project.

- Use the drop down tab to select the service interface files that you just created as the interface to use for the request and the response.
 - a. For the Request service interface field, select the .si file that you just created for the request, for example, testPhoneBookRequest.si.
 - b. For the Response service interface field, select the .si file that you just created for the response, for example, testPhoneBookResponse.si.

NOTE for response service interface use the drop to select phonebookresponse.si

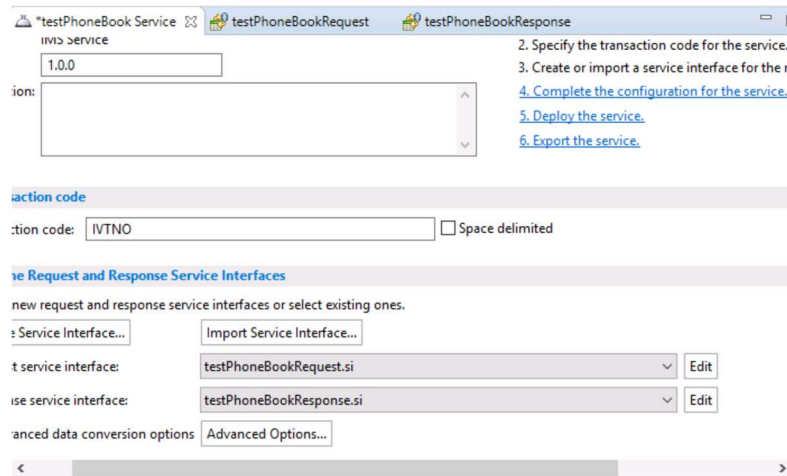


- Make sure that your request and response interfaces are the correct names Save your changes (select **File > Save**).

You have defined the service interfaces for your service. You now need to specify the IMS connection profile:

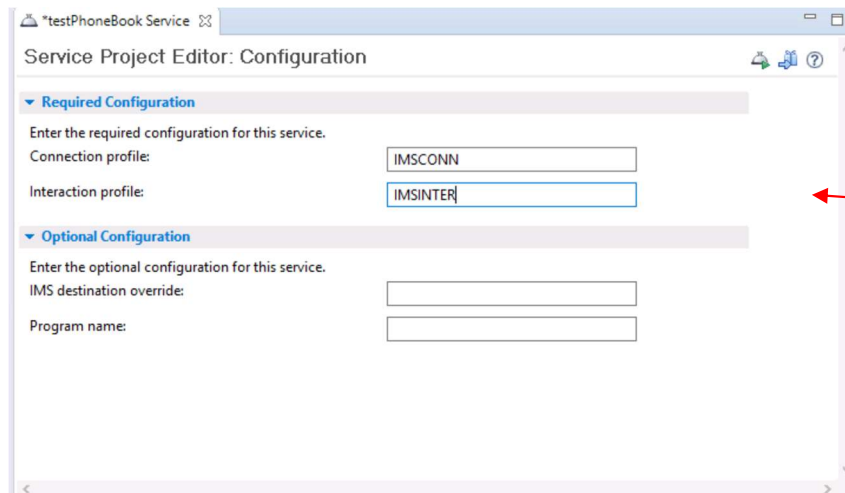
When the z/OS Connect EE server receives a RESTful service request, it first authorizes the user and then routes the request through the IMS service provider to the appropriate IMS Connect based on a defined Connection Profile. Connection profiles can be shared by any number of services.

The interaction profile which defines how the server should invoke an IMS transaction and can be shared by any number of services.



- Go back to the service project editor by clicking the tab that is named after your service project. In the service interface editor, click on ‘Complete the configuration for this service’.

In this lab we have already created Connection and Interaction profiles.



- For the Connection profile field, enter **IMSCONN**
- In the Interaction properties field, enter the **IMSINTER**

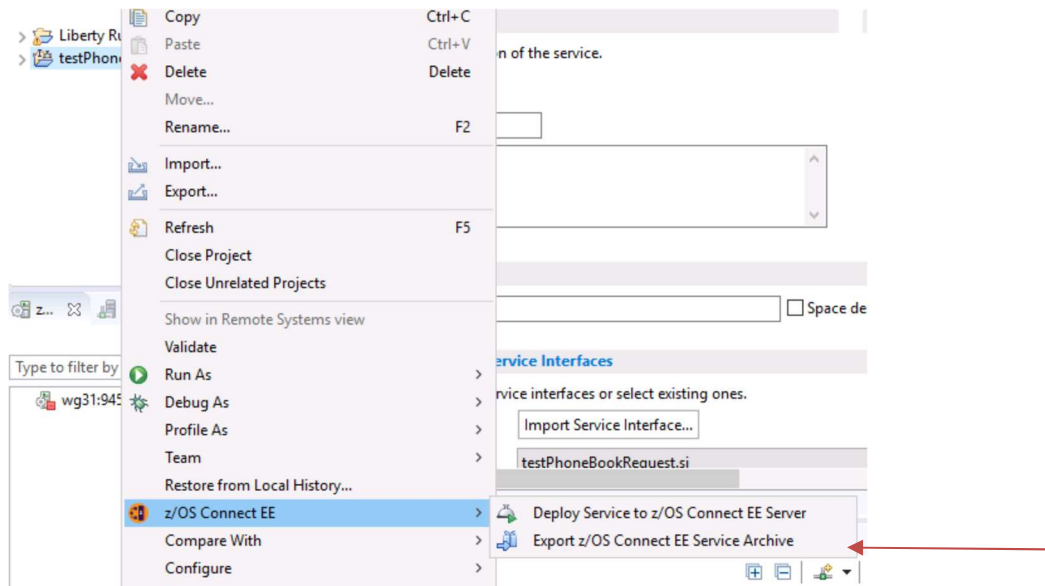
The **Optional Configuration** fields can be left blank.

- Save your changes (select **File > Save**).

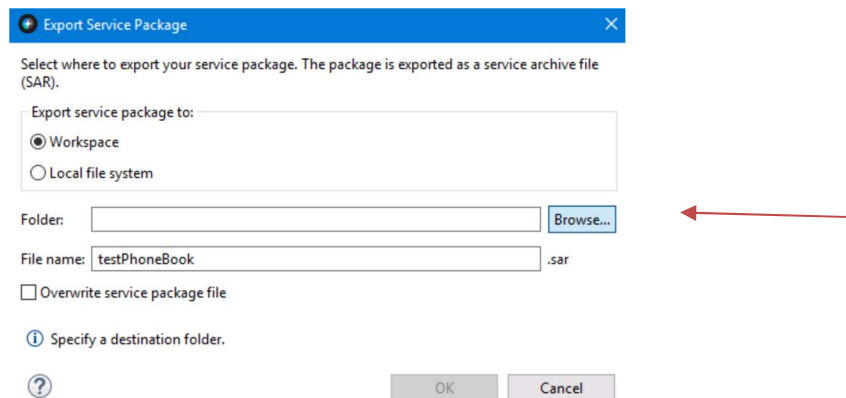
The service is now created, and all initially reported errors are gone.

You can now export the service to a service archive file to deploy it to the server.

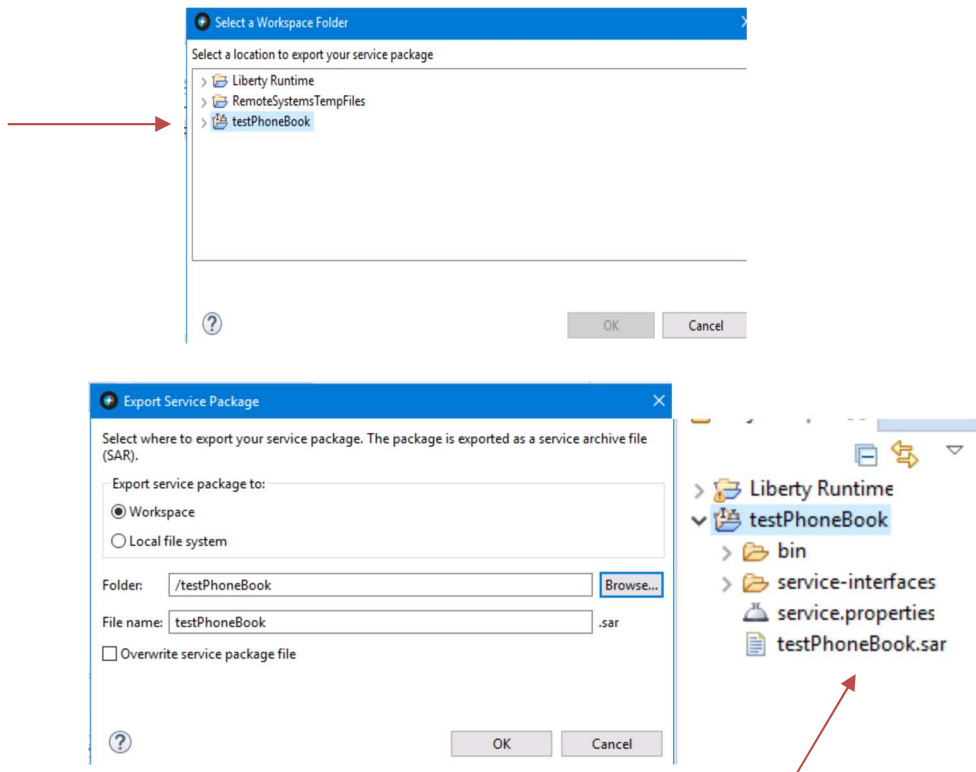
You are now ready to export your service project as a service archive file and deploy the service. **Right-click** your Project and choose **z/OS Connect EE → Export ...**



The Export Service Package opens. Choose **Workspace** and click **Browse**



Choose your project, e.g., testPhoneBook and click **OK**.

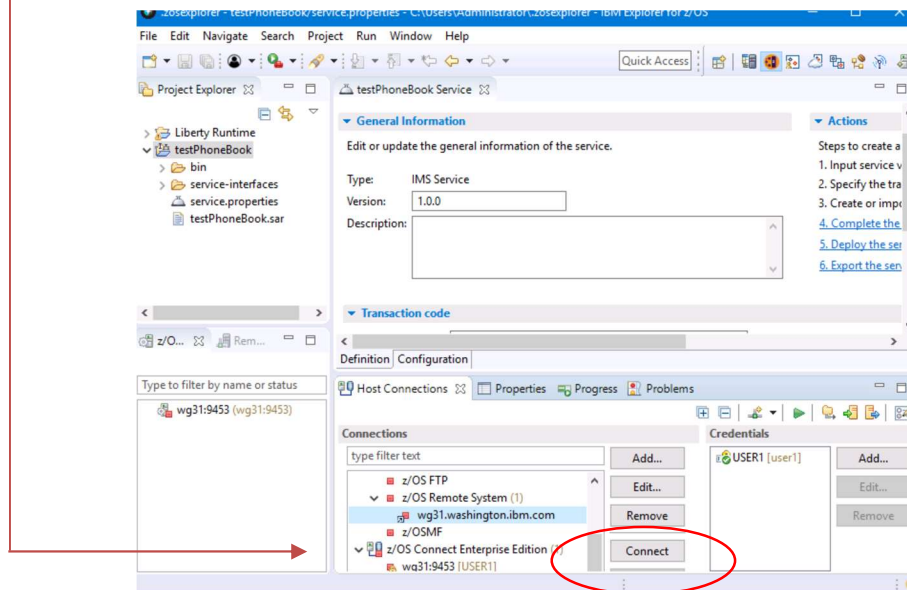


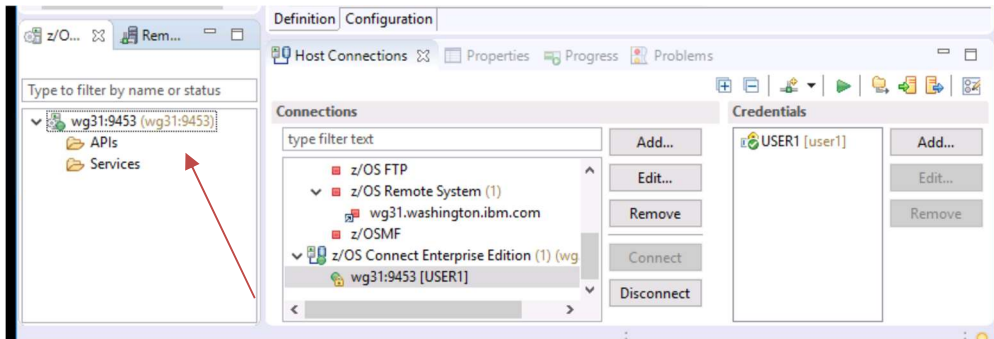
Click **OK** again. This action creates a service archive file (.sar) in your project which will

Now we need to create a connection to a z/OS Connect EE Server

In **Host Connections** view right click on z/OS Connect Enterprise Edition and select **Connect ...**

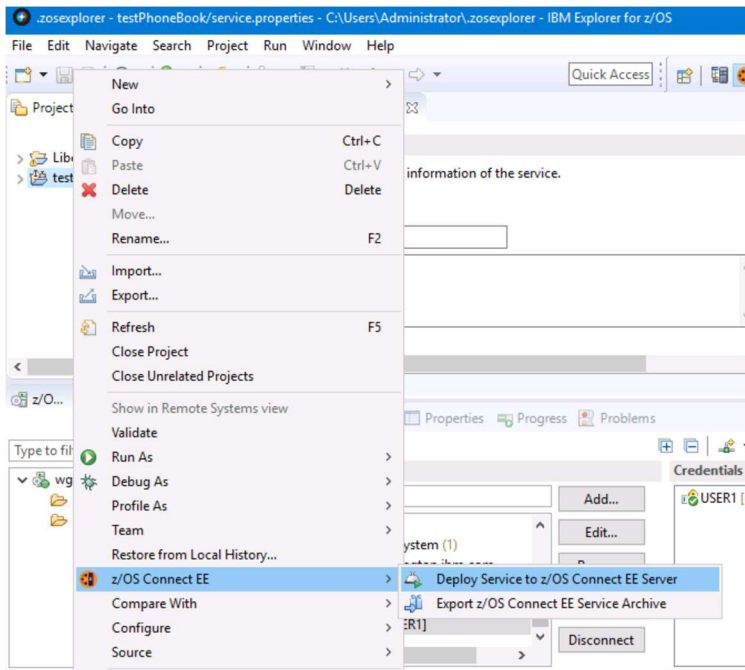
(If you do not see this panel then click **Window > Show View > Host Connections**.)



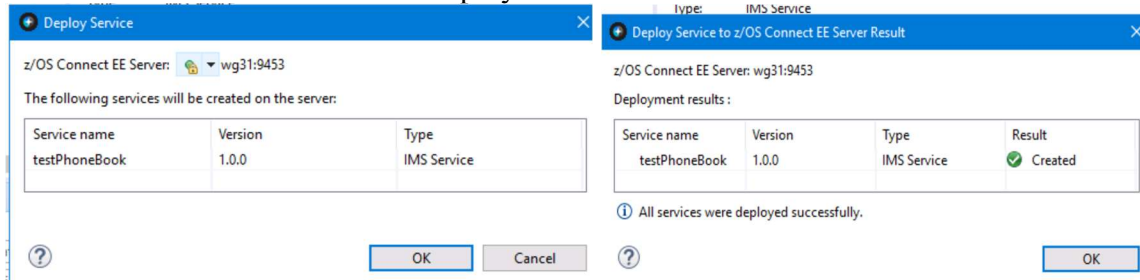


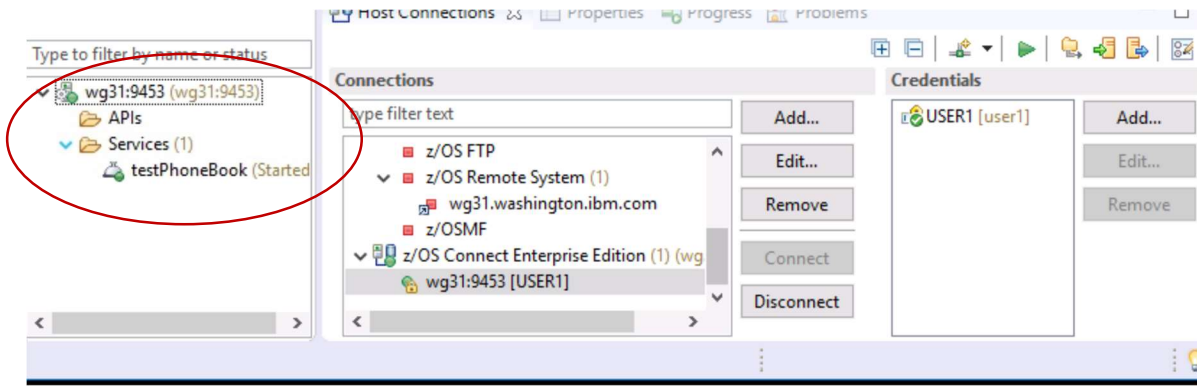
Note that you are now connected to z/OS Connect. The environment had already been set up to use **USER1** credentials.

We can now deploy the .sar file to the z/OS Connect EE Server. Right click on your project



And select z/OS Connect EE → Deploy Service to z/OS Connect Server. Click OK.



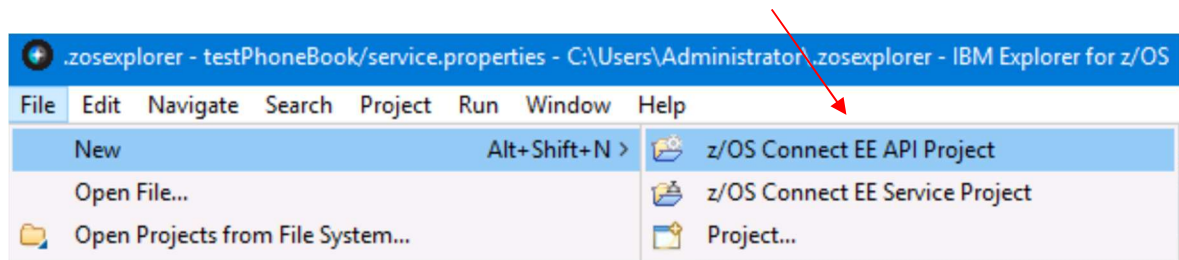


The service is now available to be called by an API.

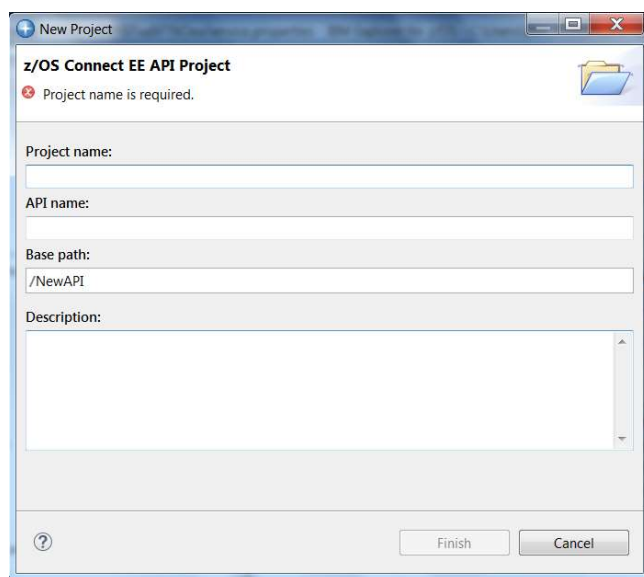
Creating an API project

Use the z/OS Connect EE API toolkit to create an API project and define the API name and base path.

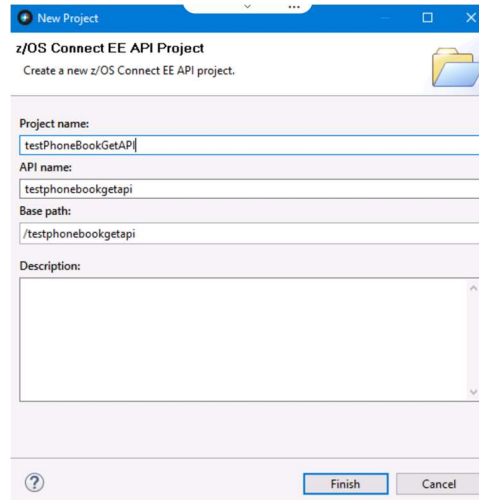
From the menu bar, select File > New > z/OS Connect EE API Project.



The z/OS Connect EE API Project wizard opens.



For project name enter **testPhoneBookGetAPI**. The API name and Base path automatically use project name. In this exercise, we will only create an API to retrieve data.



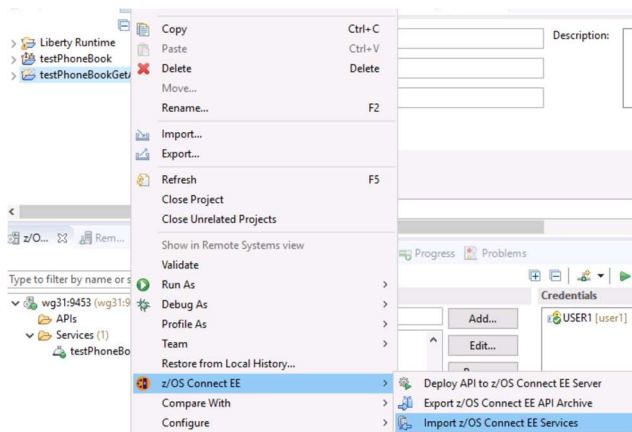
Click Finish

In this step, you will import the service archive file **testPhoneBook.sar** for the phonebook service into the API project you created in z/OS Connect EE API toolkit, and map the input and output messages for each HTTP verb. For the API design, our goal is to create the API for displaying a contact record for the phonebook DFSIVTNO transaction.

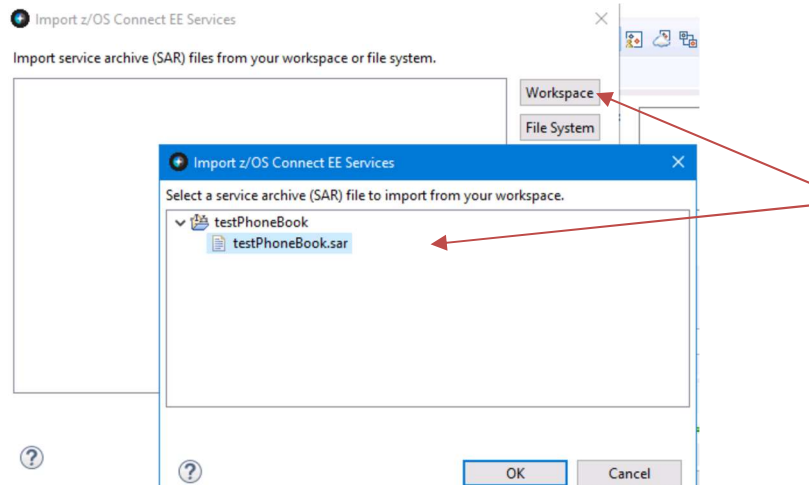
(**Optionally**, you can follow the process to create APIs for adding, updating, and deleting contacts)

Import the service archive created by z/OS Connect EE API toolkit service project.

1. In the Project Explorer view, right-click the testPhoneBookGetAPI folder and select z/OS Connect EE > Import z/OS Connect EE Services.



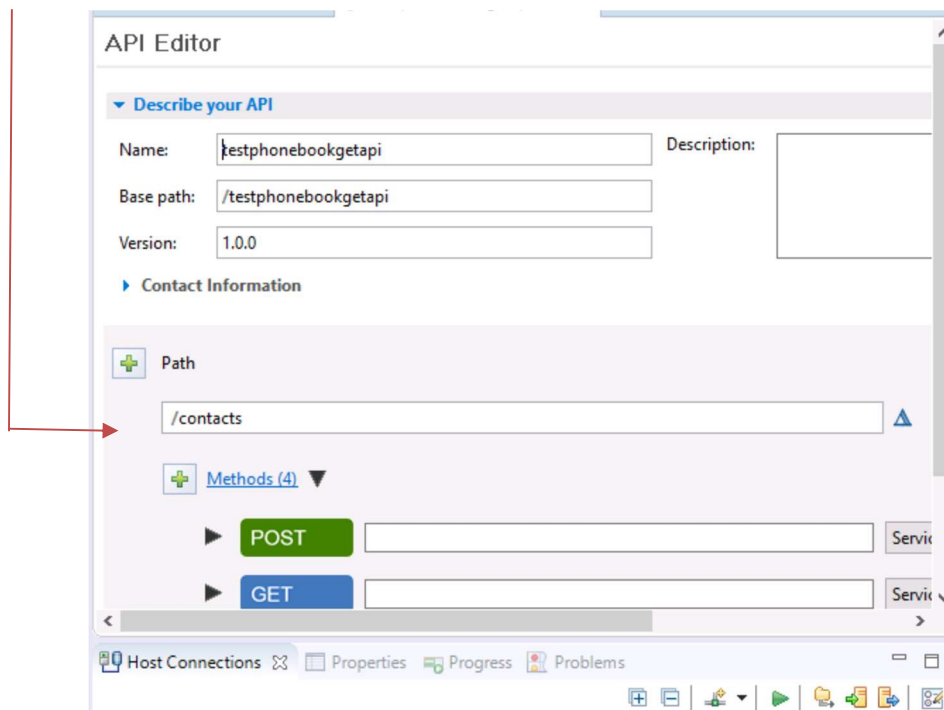
2. The Import z/OS Connect EE Services wizard opens.
3. Click **Workspace** and then select the **testPhoneBook.sar** file to import it into your API project.



4. Click OK/OK/OK.

The API project is created in the Project Explorer view. The API package.xml file opens in a tab that is named after your project. You can now model your API.

Replace **/newpath1** with **/contacts**



The base path of the API is now /testphonebookgetapi/contacts

The following APIs can be used for adding, updating, displaying, and deleting a contact record. We will only be creating an API for the GET method.

Add a contact	POST	/testphonebookgetapi/contacts
Display a contact	GET	/testphonebookgetapi /contacts/{lastName}
Update a contact	PUT	/tesphonebookgetapi /contacts/{lastName}
Delete a contact	DELETE	/testphonebookgetapi /contacts/{lastName}

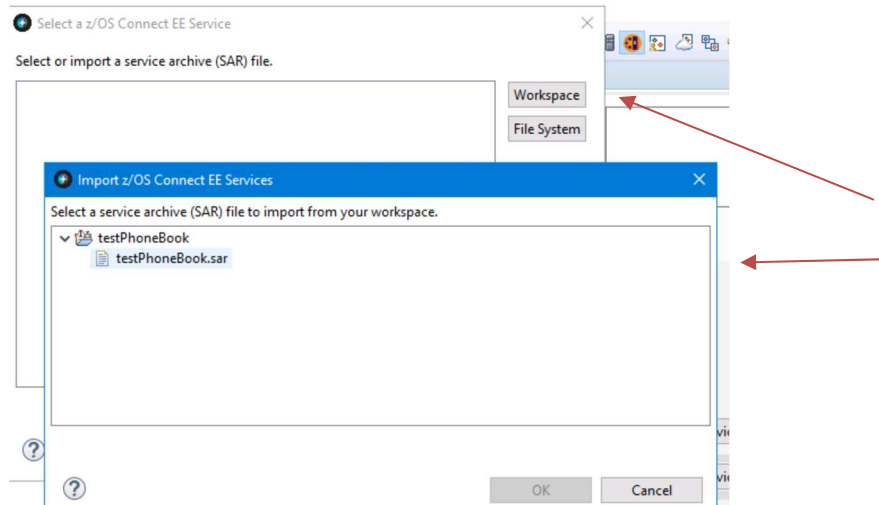
The GET method

For the GET method (querying a contact record by last name), map the data structure between the service JSON schema and the HTTP request and response messages.

Enter the API path and the path parameter, lastName, enclosed in **curly brackets** and case sensitive. In this case, it is **/contacts/{lastName}**

The screenshot shows an API management console interface. At the top, there are input fields for 'Name' (testphonebookgetapi), 'Base path' (/testphonebookgetapi), and 'Version' (1.0.0). Below these is a section titled 'Contact Information'. Under this section, there is a 'Path' field containing '/contacts/{lastName}'. Below the path field is a 'Methods (4)' section with a dropdown arrow. This section lists four HTTP methods: POST, GET, PUT, and DELETE. Each method has a corresponding 'Service' dropdown menu. A red arrow points from the text above to the 'Path' field. Another red arrow points from the text above to the 'Service' dropdown menu for the GET method.

For the GET method, click Service next to the method.

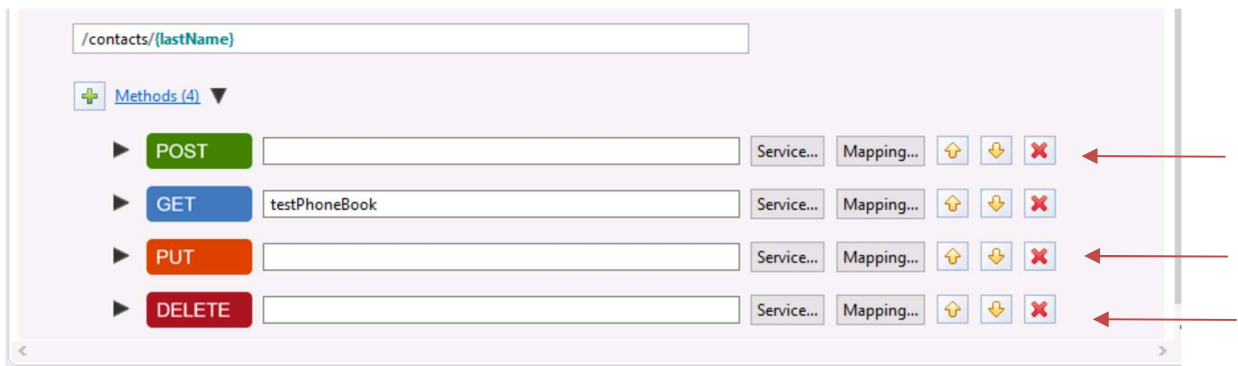


Click the **Workspace** tab in select a **z/OS Connect EE Service** panel.

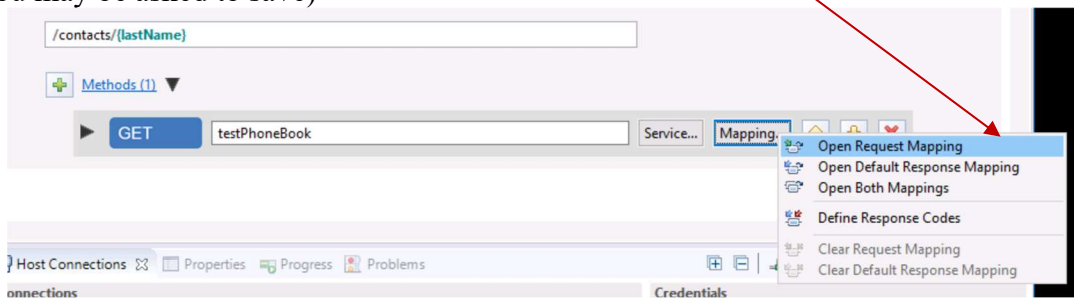
In Import z/OS Connect Service use drop down tab and select **testPhoneBook.sar** service archive file, and click OK/OK/OK.

Click File > Save or press Ctrl-S to save the service association.

Since we are only going to test the GET method in this exercise, remove the POST, PUT and DELETE methods by clicking the remove button () next to the method.

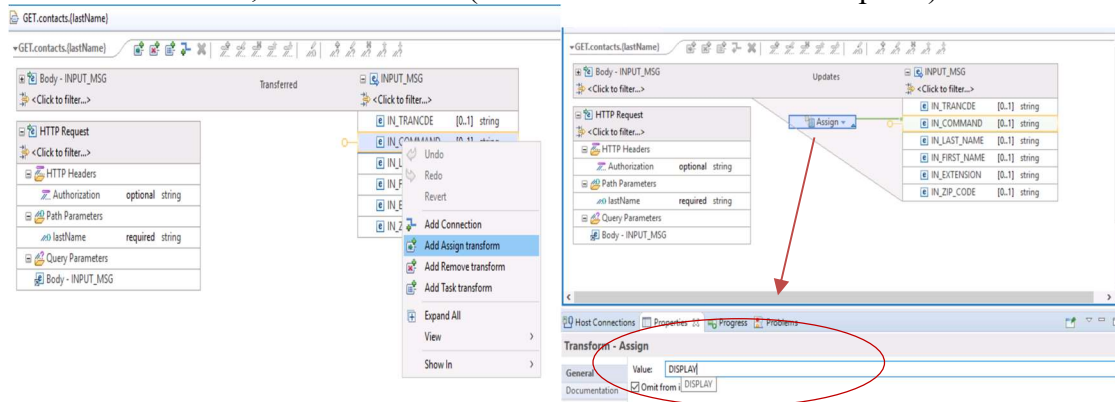


Click Mapping next to the GET method and select Open Request Mapping.
(you may be asked to save)



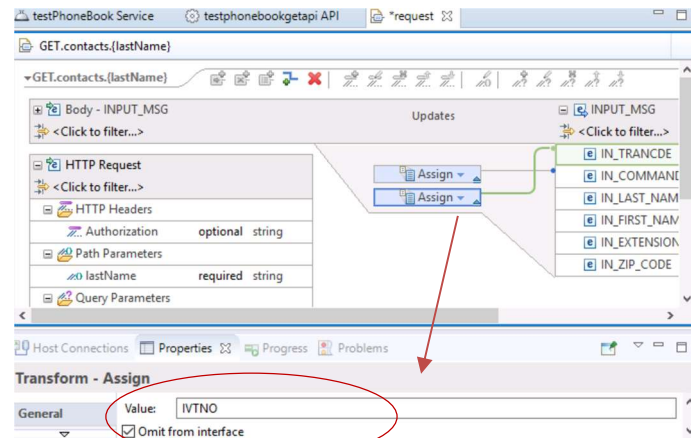
Right-click the IN_COMMAND field in the service definition on the right, and select Add Assign transform. An Assign box shows up in the middle.]

Click the Assign box, and the Transform - Assign properties view opens.
For the Value field, enter **DISPLAY**. (note that this is in the bottom panel)



You can choose to expose the transaction code in the API or assign it here (it could have also been done when we created the service).

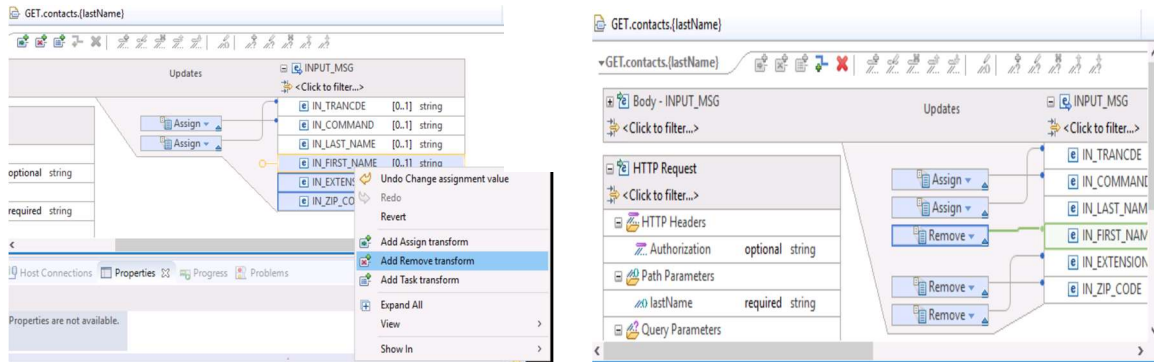
Right-click the IN_TRANCDE field and choose Add Assign transform. An Assign box shows up in the middle. Click the Assign box, and the Transform - Assign properties view opens. For the Value field, enter **IVTNO**. (note that this is in the bottom panel)



Because only the last name is needed for the request and the other fields are not needed, remove those fields so they are hidden from the client.

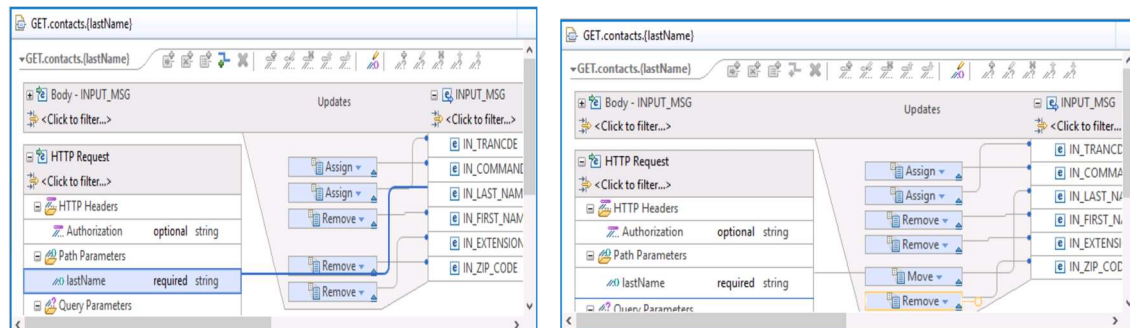
Hold down the Ctrl key to select multiple fields: IN_EXTENSION, IN_FIRST_NAME, and IN_ZIP_CODE. Right-click and select Add Remove transform.

This removes unneeded fields from the requests for this method.



The last step is to map the path parameter lastName from the HTTP Request section to the IN_LAST_NAME field.

Click and drag the lastName path parameter field in the HTTP Request section to the IN_LAST_NAME field on the right.



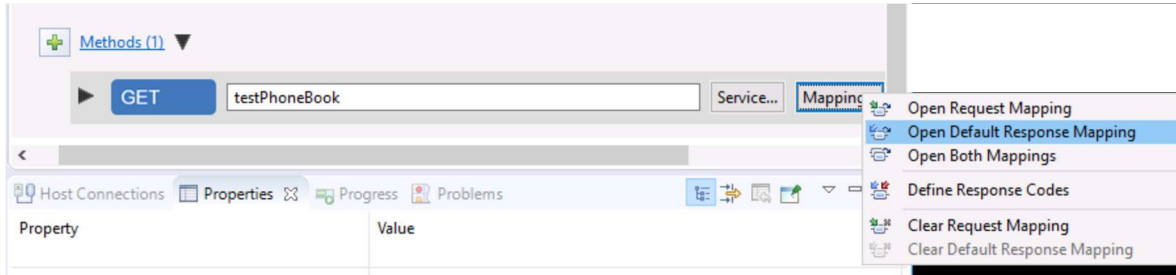
No JSON body is sent in with this request. All the information needed to display a contact record is carried in on the URI with the path parameter. Note that the actions include:

2 Assigns, 3 Removes, and 1 Move.

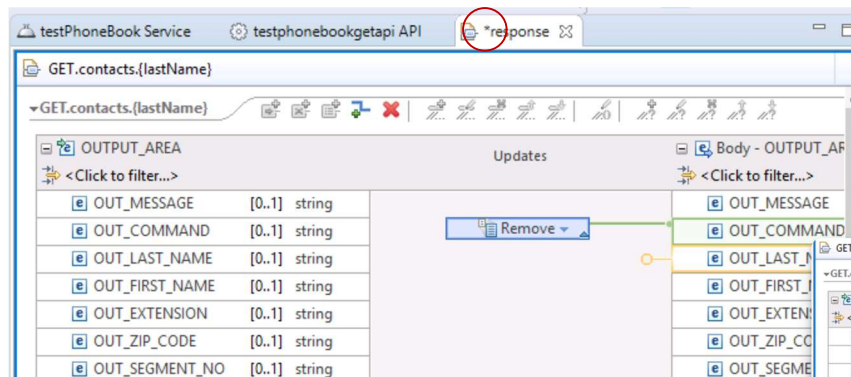
Click **File > Save (Ctrl-S)** to save your request mapping. Close the request mapping tab.

The next step is to map the response. For the given last name, we want to return all fields except the OUT_COMMAND field, so we need to remove this field from the response.

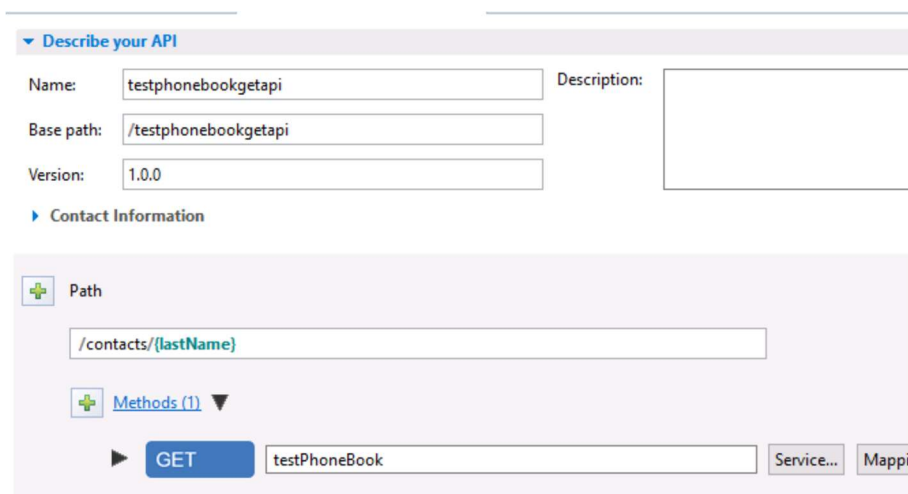
Click Mapping next to the GET method and select Open Response Mapping.



Right-click OUT_COMMAND on the right, and select Add Remove transform. This removes the OUT_COMMAND field from the response.

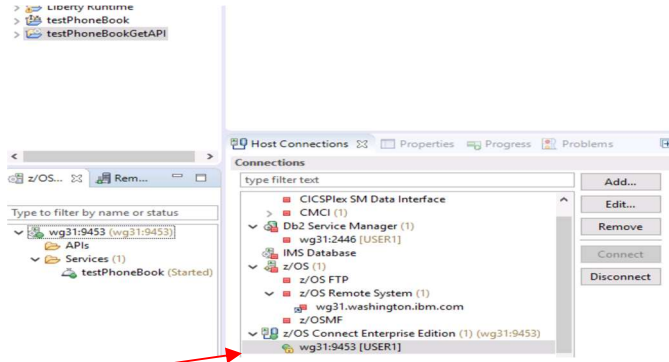


Click **File > Save (Ctrl-S)** to save your response mapping and close the response tab. (Note: The tool indicates that something has not been saved with an asterisk *)



Deploying the API

To Deploy the API you will use the same z/OS Connect EE Server connection created for deploying the service archive file(sar).

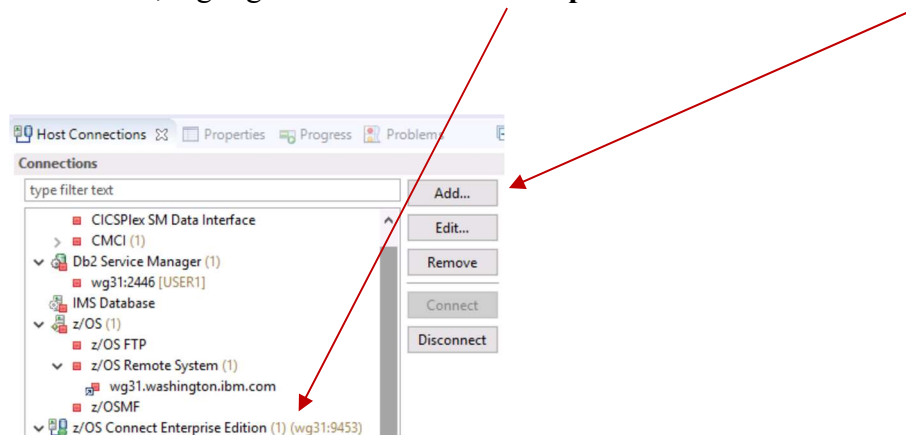


In the **Host Connections** view, check to see if you are still connected to the z/OS Connect server. Otherwise, click Add and select z/OS Connect Enterprise Edition.

If you do not see this panel then click **Window > Show View > Host Connections**.

You may still have the connection to the host from the previous step where you deployed the service. If so, move to the instruction for **Deploy your API**. If not, the following steps establish the connection.

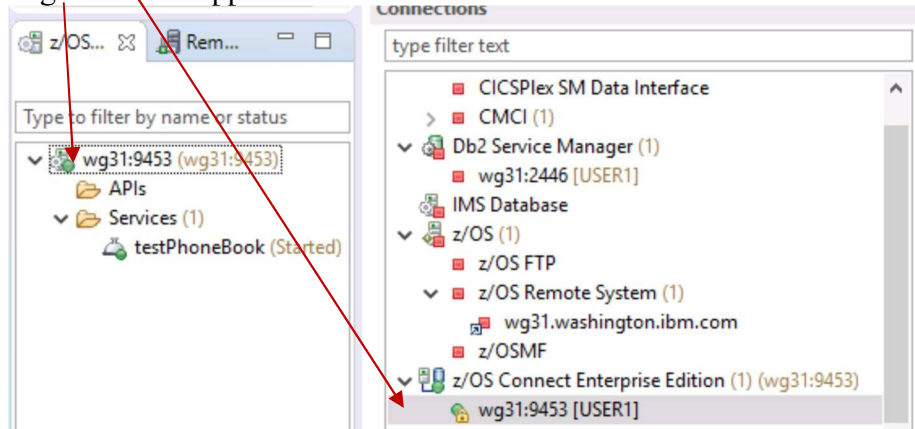
In the **Connections** list, highlight **z/OS Connect Enterprise Edition** and click **Add**.



Host name – WG31 Port number – 9453

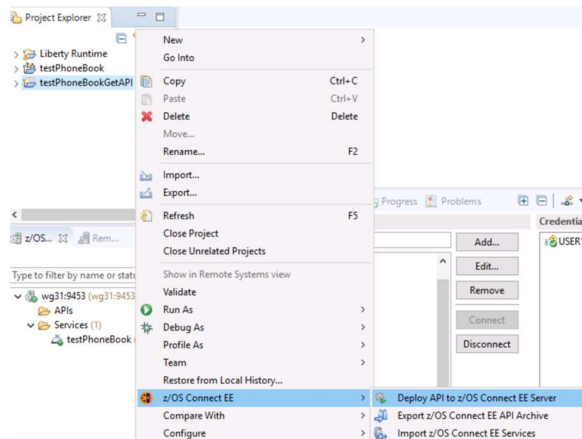
Save and Connect. If you are asked for a userid, user USER1 and password USER1.

A green token appears when the connection is established.



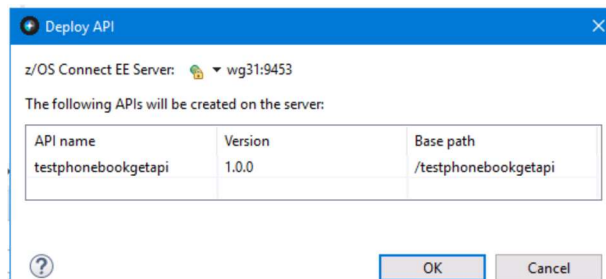
To deploy your API:

In the Project Explorer view, highlight your API project and right-click



Select z/OS Connect EE > Deploy API to z/OS Connect EE Server.

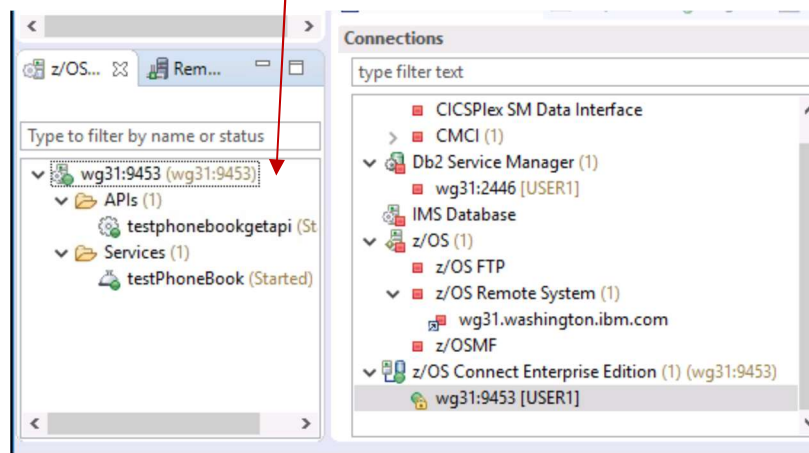
In the Deploy API window, select the server to which to deploy the API.



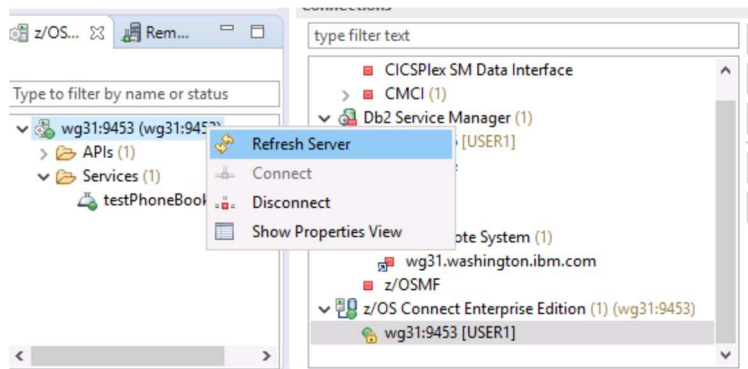
Click OK. Deployment result is reported.

A deployed API is automatically started unless specified otherwise in the z/OS Connect EE preferences window

Check to see that your API appears in the server's list

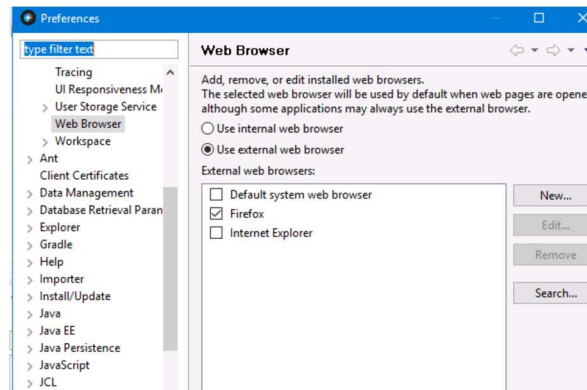


You might need to refresh the server view in order to see your deployed API

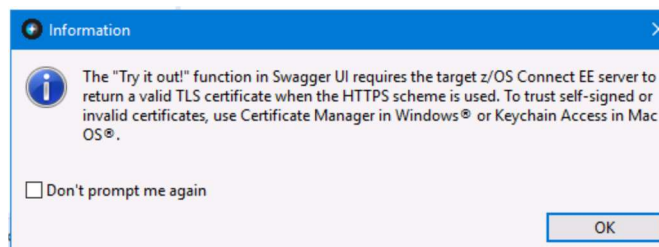
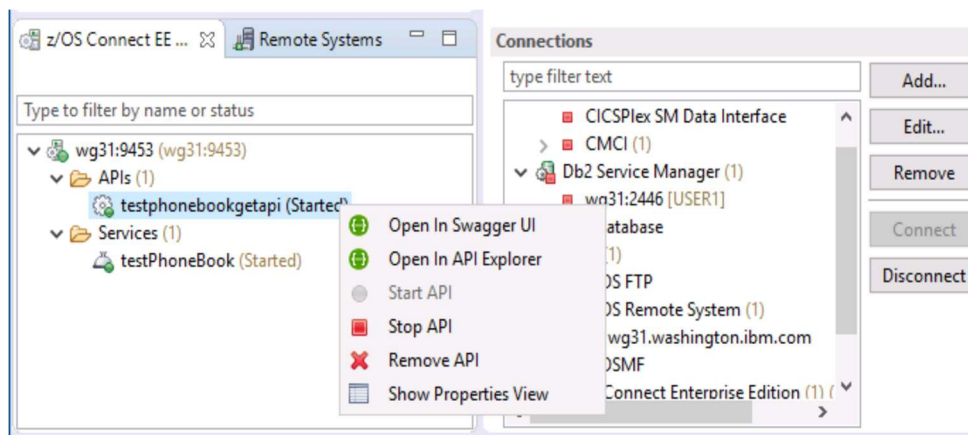


Testing the API

You can test your API directly from z/OS Connect EE API toolkit by using the "Try it out!" function in Swagger UI. ***Make sure that you first choose an external browser for testing.*** This has already been done for you but if you want to check, then click on the following: Window > Preferences > General > Web Browser



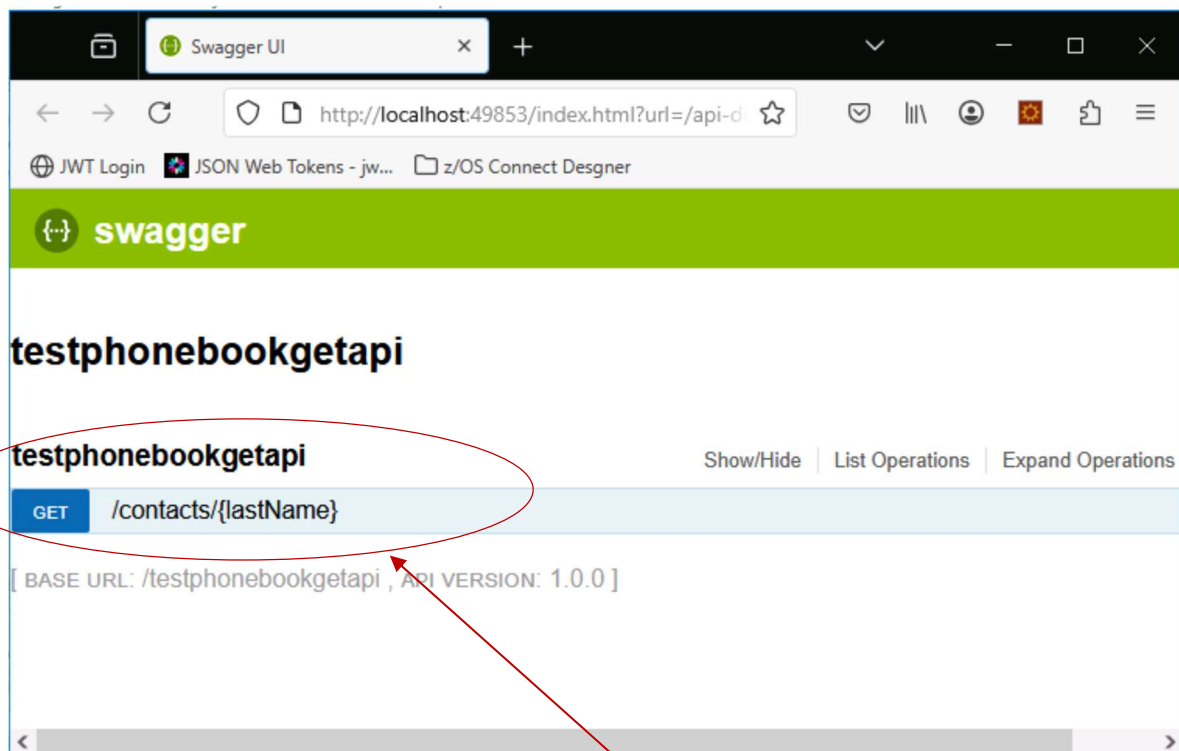
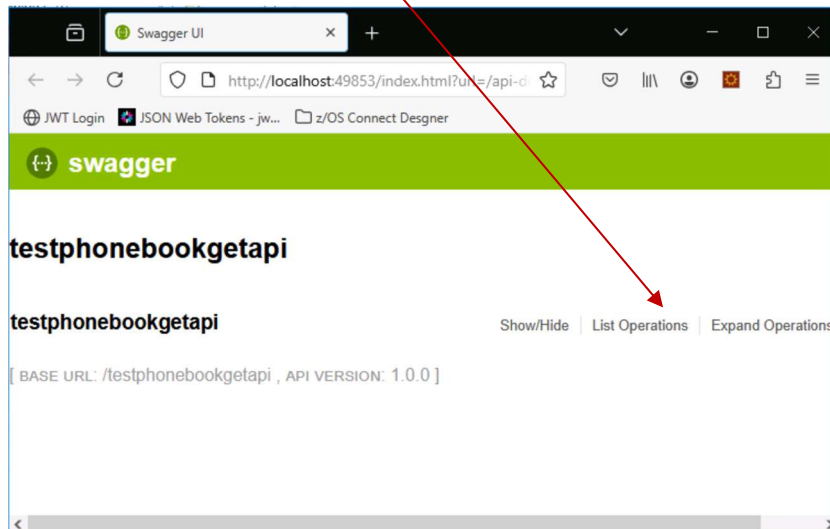
Go to the server window and **right-mouse Click** on your API and select Open in Swagger UI.



Click OK.

A browser window opens up.

Click List Operations to see available operations in the API.

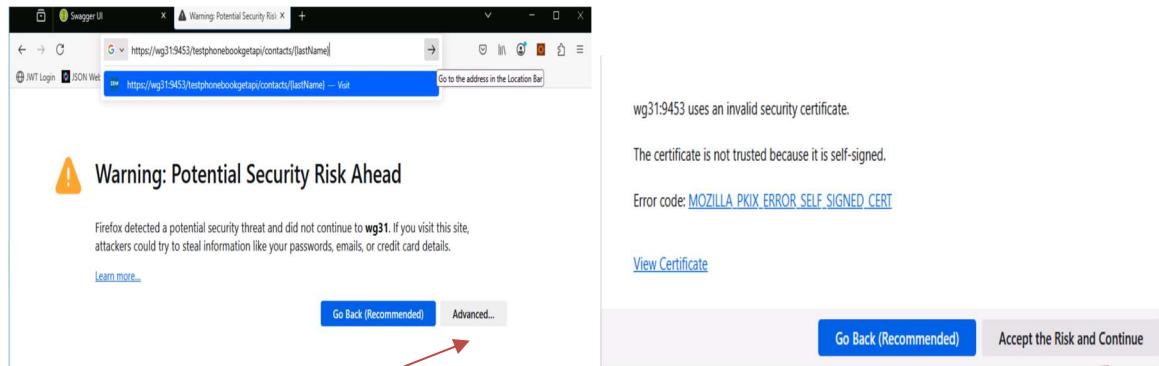


Note the base URL for this api is : testphonebookgetapi/contacts/{lastName}

ALSO note that for this environment **BEFORE** running this {...}swagger, we will first need to request a certificate from the server environ,ent.

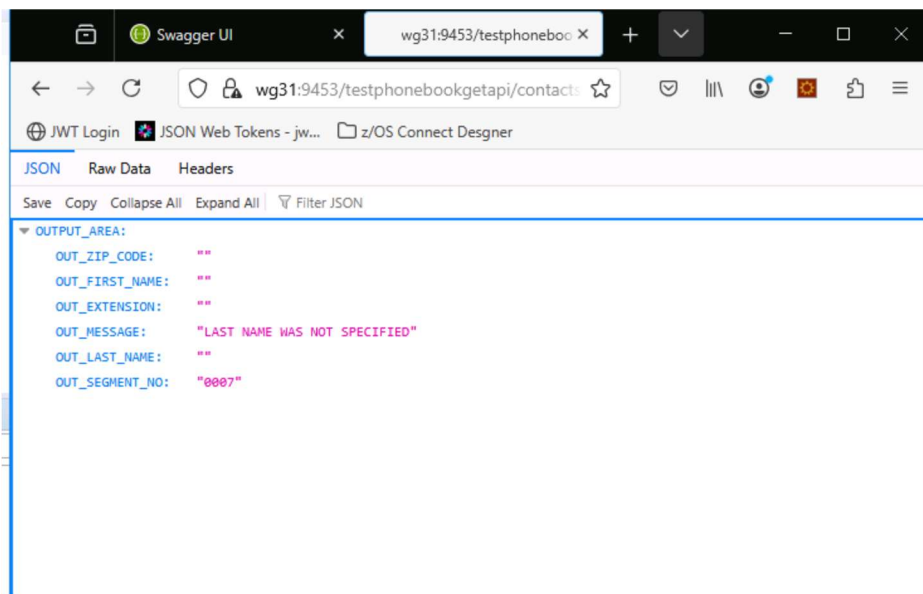
Open up another browser window in the same browser as your swagger UI and key in **https://wg31:9453/testphonebookgetapi/contacts/**

where wg31 is the z/OS host, 9453 is the port number of the z/OS Connect server we are using, and **/testphonebookgetapi/contacts/** is the url for you API. Note that if you used different names then those will be what you use.



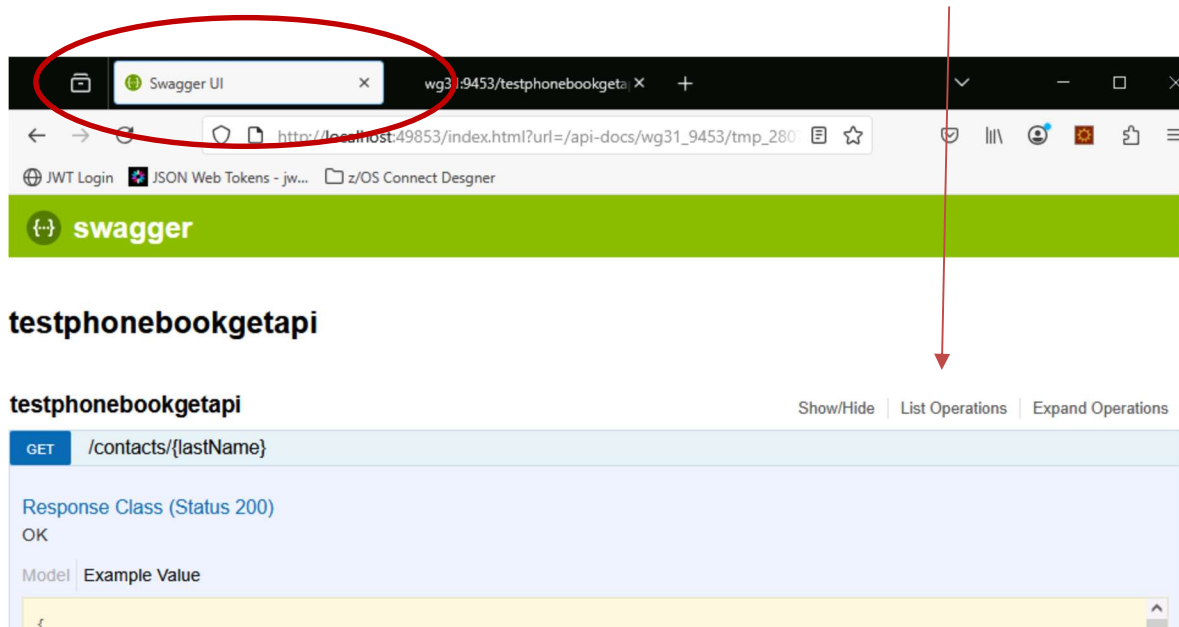
Click **Advanced**.

Scroll to the bottom of the window and **Accept the Risk**



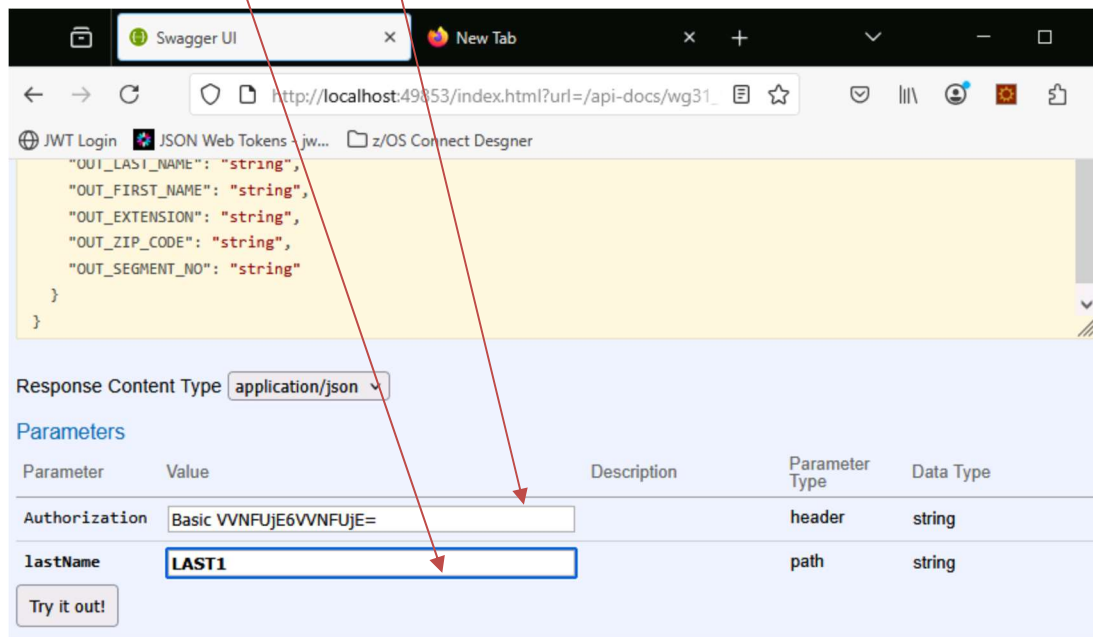
The request will be sent to the server and since we didn't specify a last name, the message returned is "LAST NAME WAS NOT SPECIFIED" The important point is that this browser access with a certificate can be used by the Swagger UI.

We are now ready to switch back to the Swagger UI window. Click on **List Operations** and the **GET** method.



Since the server needs authorization credentials. The one you will use is an encoded version of the USER1 id and USER1 password. Key in:

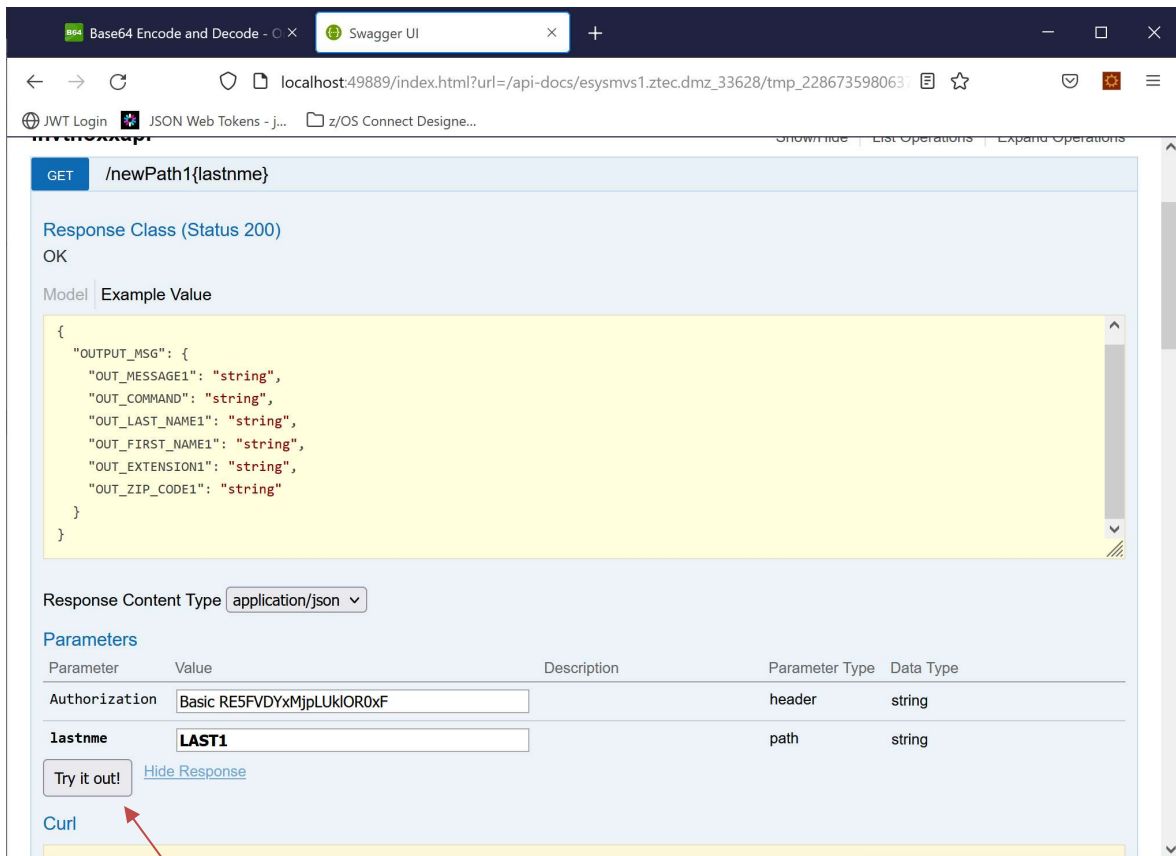
Basic VVNFUJE6VVNFUjE= (note a separate document will detail how to encode)
lastName: LAST1



BASE URL: /testphonebookgetapi API VERSION: 1.0.0

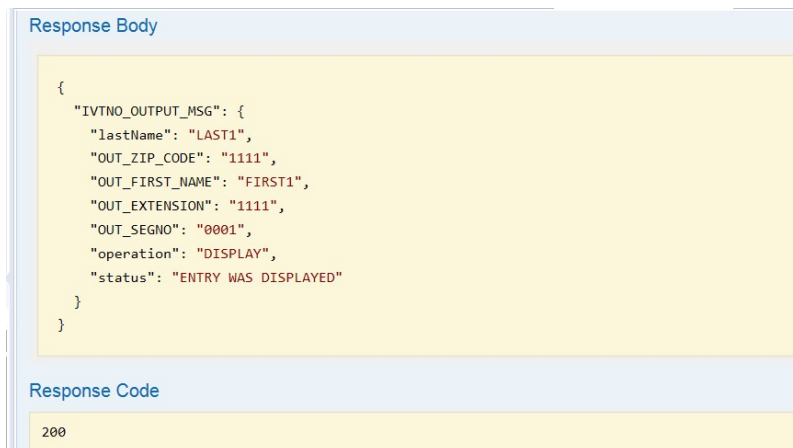
After you click Get, the browser opens up the parameter list. (For the GET api, you only had one parameter /contacts/{lastName} so this is what the Swagger UI will ask you to input.

Key in: LAST1 in the lastName field (alternatively you could try MOUSE or SHARE)
For authorization, Key in: Basic xxxxx where xxxxx is the value from your encoded request.



Then select the **Try it out!** tab

The Response body displays the information in the database associated with the lastName.



You can also try out a last name of MOUSE.

Congratulations you have completed this lab.

The next lab will walk you through the creation of a single API (invoking the same transaction) but with all the methods: GET, PUT, POST, DELETE